MCA Semester III

Advance Java Programming (Spring Boot)

Practical Assignment — Worked Solutions (Practicals 1-20)

Chapters 5-7 — JPA, RESTful Microservices & Security with H2 Database

Each practical includes: Objective • Class/ER Diagram • Fully Commented Code • Step-by-Step Instructions • Postman API Testing

Before You Start: One-Time Project Setup

All 20 practicals live inside a single Spring Boot project so entities you build in one practical (e.g. Employee) can be reused by later practicals. Each practical gets its own Java sub-package (p1, p2, p3 ...) so identically-named classes in different practicals never collide.

Step 1 — Create the Spring Boot Project

1. Go to https://start.spring.io (Spring Initializr).

2. Project: Maven, Language: Java, Spring Boot: 3.3.x

3. Group: com.exam Artifact: spring-practical-exam Packaging: Jar Java: 17

4. Add dependencies: Spring Web, Spring Data JPA, H2 Database, Validation, Spring Security, OAuth2 Resource Server, OAuth2 Client, Lombok.

5. Click Generate, unzip the project, and open it in IntelliJ IDEA / Eclipse / VS Code.

> *i You can also add dependencies later by editing pom.xml directly (shown below) instead of re-generating the project.*

Step 2 — pom.xml (all dependencies used across Practicals 1-20)

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/>
    </parent>

    <groupId>com.exam</groupId>
    <artifactId>spring-practical-exam</artifactId>
    <version>1.0.0</version>
    <name>spring-practical-exam</name>
    <description>MCA Sem III - Spring Boot Practical Exam (Practicals 1-20)</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Core REST + MVC support (Chapter 6) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring Data JPA (Chapter 5) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- H2 in-memory database - no server install needed for the exam -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
```

```
        <!-- OPTIONAL: swap H2 for real MySQL by un-commenting this
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        -->

        <!-- Bean Validation - needed from Practical 13 onward -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- Spring Security - needed from Practical 15 onward -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

        <!-- OAuth2 Resource Server (JWT) - needed for Practical 18 -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
        </dependency>

        <!-- OAuth2 Client (Google/GitHub login) - needed for Practical 19 -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-oauth2-client</artifactId>
        </dependency>

        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Step 3 — application.properties

This project uses the H2 in-memory database so no separate database server installation is required. If your college specifically requires MySQL, swap the MySQL dependency into pom.xml and replace the datasource lines with your MySQL URL/username/password — every entity/repository/controller written in this document works unchanged either way.

src/main/resources/application.properties

```
# ==== Server ====
```

```
server.port=8080

# ==== H2 in-memory datasource ====
spring.datasource.url=jdbc:h2:mem:examdb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# ==== JPA / Hibernate ====
# ddl-auto=update -> Hibernate creates/updates tables from @Entity classes automatically
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# ==== H2 web console (visit http://localhost:8080/h2-console) ====
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Step 4 — Recommended Package Layout

Project structure

```
spring-practical-exam
 └── src/main/java/com/exam
      ├── SpringPracticalExamApplication.java
      ├── p1/   Employee, EmployeeRepository, EmployeeRestController
      ├── p2/   Product, ProductRepository, ProductRestController
      ├── p3/   (reuses p1.Employee) EmployeeRepository (custom queries)
      ├── p4/   User, UserProfile, Employee, Passport, Country, Capital ...
      ├── p5/   Department, Employee, DepartmentRepository, DepartmentRestController
      ├── p6/   Student, Course ...
      ├── ...   (one sub-package per practical, p6 - p20)
      └── p20/  CorsConfig ...
 └── src/main/resources/application.properties
 └── pom.xml
```

Step 5 — Running the Application

1. From your IDE, right-click SpringPracticalExamApplication.java → Run.

2. Or from a terminal in the project folder, run: mvnw spring-boot:run (Windows) or ./mvnw spring-boot:run (Mac/Linux).

3. Confirm the console shows: Tomcat started on port(s): 8080 and Started SpringPracticalExamApplication.

4. Open http://localhost:8080/h2-console in a browser to inspect tables at any time (JDBC URL: jdbc:h2:mem:examdb).

5. Open Postman (download from https://www.postman.com/downloads/ if not already installed) — this is what we use to test every REST endpoint below.

Chapter 5

Working with Spring Data JPA

Practical No. 1 — Introduction to JPA: Entity Class and Persisting a Single Entity

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue), configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's CrudRepository.

Design — Class Diagram

The flow below is the pattern every practical in this document follows: an @Entity class is mapped to a table, a Repository interface gives us ready-made database operations, and a @RestController exposes those operations over HTTP for Postman (or any client) to call.



Step-by-Step Implementation

Step 1 — Add the dependencies (already done in the common pom.xml)

spring-boot-starter-data-jpa and the H2 driver were added in the one-time setup section above. spring-boot-starter-data-jpa auto-configures a DataSource, an EntityManagerFactory and transaction management for us.

Step 2 — Configure the datasource (already done in application.properties)

spring.jpa.hibernate.ddl-auto=update tells Hibernate: read every @Entity class on startup and create or alter the matching table automatically, so we never hand-write CREATE TABLE statements during development.

Step 3 — Create the Employee entity

Create the file com.exam.p1.Employee. Key annotations:

- @Entity — marks the class as a JPA-managed table.
- @Id — marks empId as the primary key column.
- @GeneratedValue(strategy = GenerationType.IDENTITY) — the database itself auto-increments the id on insert.

src/main/java/com/exam/p1/Employee.java

```
package com.exam.p1;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

/**
 * Employee entity.
 * @Entity tells Hibernate this class maps to a database table ("employee").
 * A no-arg constructor is required by JPA so Hibernate can instantiate the
 * object via reflection before populating its fields.
 */
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // DB auto-increments empId
    private int empId;

    private String empName;
    private String department;
    private double salary;

    // ---- No-arg constructor (REQUIRED by JPA) ----
    public Employee() {
    }

    // ---- Convenience constructor for creating objects in code ----
    public Employee(String empName, String department, double salary) {
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    // ---- Getters and Setters (JPA + Jackson JSON binding need these) ----
    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
```

```
        this.department = department;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

Step 4 — Create EmployeeRepository

This interface needs no method bodies — Spring Data JPA generates the implementation at runtime using a dynamic proxy.

src/main/java/com/exam/p1/EmployeeRepository.java

```
package com.exam.p1;

import org.springframework.data.repository.CrudRepository;

/**
 * EmployeeRepository
 * Extending CrudRepository<Employee, Integer> instantly gives us, with
 * ZERO implementation code, the following methods (Spring generates a
 * proxy implementation at runtime):
 *   save(Employee)        -> INSERT or UPDATE
 *   findAll()              -> SELECT * FROM employee
 *   findById(Integer)      -> SELECT * FROM employee WHERE emp_id = ?
 *   deleteById(Integer)    -> DELETE FROM employee WHERE emp_id = ?
 *   count(), existsById(), ... etc.
 *
 * Generic types: <Employee, Integer> = <Entity type, Primary-key type>
 */
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
}
```

Step 5 & 6 — Create EmployeeRestController with full CRUD

@RestController combines @Controller + @ResponseBody, so every method's return value is automatically serialized to JSON. @RequestMapping("/api/employees") sets the common base path for all endpoints in this class.

src/main/java/com/exam/p1/EmployeeRestController.java

```
package com.exam.p1;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

/**
 * EmployeeRestController
 * Exposes the Employee entity over HTTP as a REST API.
 * Base path for every method below: /api/employees
 */
@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
```

```
    private EmployeeRepository employeeRepository;

    // ---- CREATE : POST /api/employees ----
    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        // save() inserts a new row and returns the row WITH the generated empId
        return employeeRepository.save(employee);
    }

    // ---- READ ALL : GET /api/employees ----
    @GetMapping
    public List<Employee> getAllEmployees() {
        // CrudRepository.findAll() returns Iterable<Employee>; wrap into a List
        List<Employee> list = new java.util.ArrayList<>();
        employeeRepository.findAll().forEach(list::add);
        return list;
    }

    // ---- READ ONE : GET /api/employees/{id} ----
    @GetMapping("/{id}")
    public Employee getEmployeeById(@PathVariable int id) {
        Optional<Employee> employee = employeeRepository.findById(id);
        return employee.orElse(null); // Practical 14 replaces this with a proper 404
    }

    // ---- UPDATE : PUT /api/employees/{id} ----
    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable int id, @RequestBody Employee updated) {
        // 1) fetch the existing row
        Optional<Employee> existingOpt = employeeRepository.findById(id);
        if (existingOpt.isEmpty()) {
            return null;
        }
        // 2) modify its fields
        Employee existing = existingOpt.get();
        existing.setEmpName(updated.getEmpName());
        existing.setDepartment(updated.getDepartment());
        existing.setSalary(updated.getSalary());
        // 3) save() again -> id already exists, so Hibernate runs UPDATE, not INSERT
        return employeeRepository.save(existing);
    }

    // ---- DELETE : DELETE /api/employees/{id} ----
    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable int id) {
        employeeRepository.deleteById(id);
        return "Employee deleted successfully";
    }
}
```

Step 7 — Run and verify

1. Run SpringPracticalExamApplication. Watch the console log — Hibernate prints "create table employee (...)" the first time it runs.

2. Open http://localhost:8080/h2-console, connect with JDBC URL jdbc:h2:mem:examdb, and confirm an EMPLOYEE table now exists (it will be empty).

3. Use Postman (next section) to insert a record with POST, then refresh the H2 console and run SELECT * FROM EMPLOYEE to see the row physically stored.

Postman API Testing

With the application running on port 8080, open Postman and test each endpoint below, in order. Set the Body tab to raw / JSON for POST and PUT requests.

Practical 1 — Postman API Testing (Employee CRUD)

▸ 1. POST — create a new employee

POST http://localhost:8080/api/employees Send

Body > raw (JSON)

```
{
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

Response Status: 200 OK

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

▸ 2. GET — fetch all employees

GET http://localhost:8080/api/employees Send

Response Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  }
]
```

▸ 3. GET — fetch employee by id

GET http://localhost:8080/api/employees/1 Send

Response Status: 200 OK

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 55000.0
}
```

▸ 4. PUT — update employee salary

PUT http://localhost:8080/api/employees/1 Send

Body > raw (JSON)

```
{
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 62000.0
}
```

Response Status: 200 OK

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "department": "IT",
  "salary": 62000.0
}
```

▸ 5. DELETE — remove employee

DELETE http://localhost:8080/api/employees/1 Send

Response Status: 200 OK

```
{
  "message": "Employee deleted successfully"
}
```

> ***i*** *Tip: after the POST call returns empId: 1, reuse that same id (1) in the GET-by-id, PUT and DELETE calls that follow.*

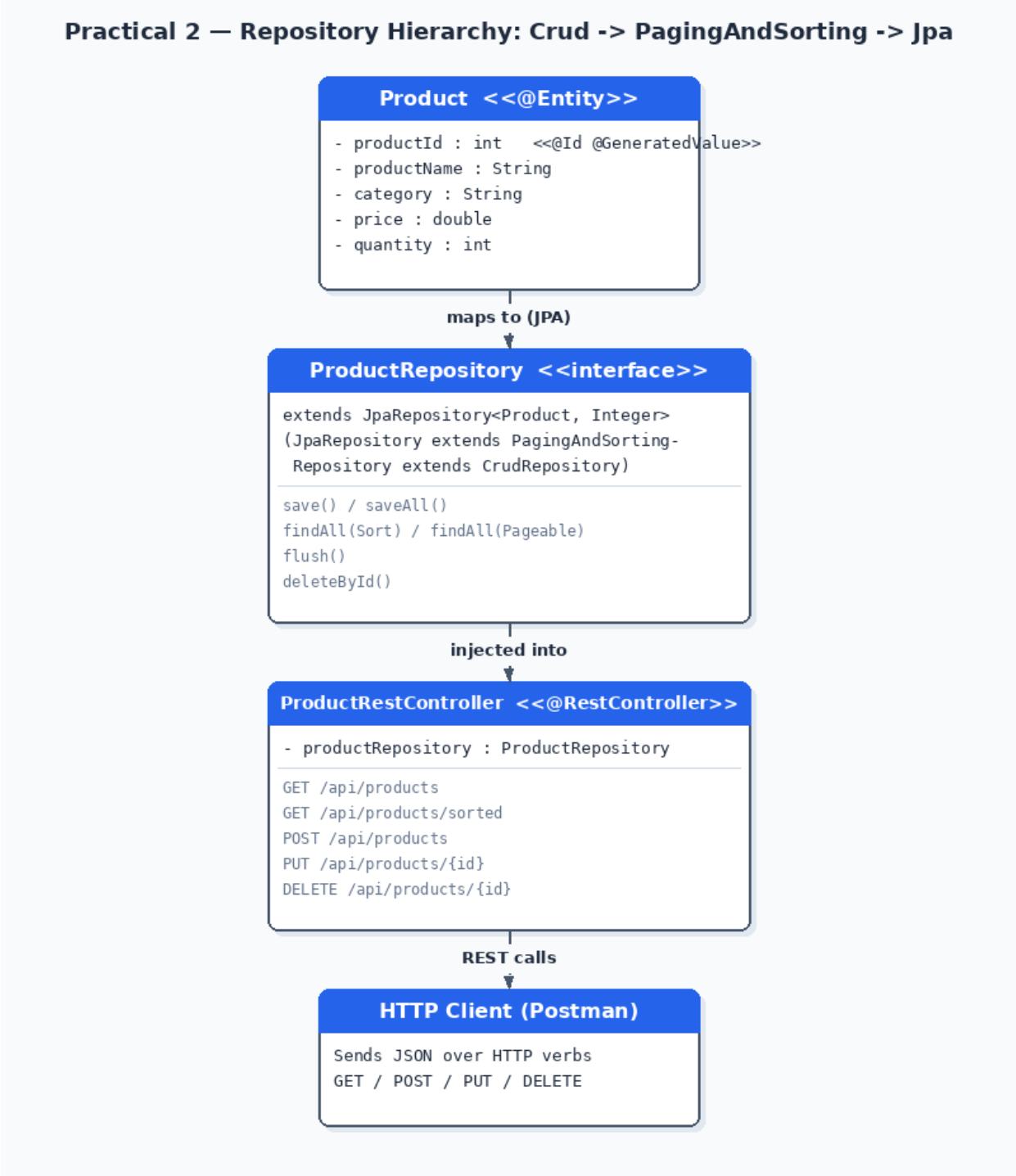
Practical No. 2 — Spring Data Repository Hierarchy: CrudRepository vs PagingAndSortingRepository vs JpaRepository

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To compare the three levels of the Spring Data repository hierarchy — CrudRepository (basic CRUD), PagingAndSortingRepository (adds paging/sorting), and JpaRepository (adds JPA-specific batch operations) — by rewriting the same repository at each level and observing the additional methods available.

Design — Class Diagram



Step-by-Step Implementation

Step 1 — Create the Product entity

src/main/java/com/exam/p2/Product.java

```
package com.exam.p2;


import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
    }

    public Product(String productName, String category, double price, int quantity) {
        this.productName = productName;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    // Getters and Setters
    public int getProductId() { return productId; }
    public void setProductId(int productId) { this.productId = productId; }
    public String getProductName() { return productName; }
    public void setProductName(String productName) { this.productName = productName; }
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

Step 2 — Stage 1: extend CrudRepository

Write this first and expose save/findAll endpoints. At this stage only basic CRUD methods are available — there is no way to sort or page results.

ProductRepository.java (Stage 1)

```
// ----- STAGE 1 : CrudRepository -----
// Gives only basic CRUD: save(), findAll(), findById(), deleteById(), count()
public interface ProductRepository extends CrudRepository<Product, Integer> {
}
```

Step 3 — Stage 2: change to PagingAndSortingRepository

Simply change the extends clause. Immediately, findAll(Sort) and findAll(Pageable) become available on the same interface — no other code changes.

ProductRepository.java (Stage 2)

```
// ----- STAGE 2 : PagingAndSortingRepository -----
// Adds on top of CrudRepository:
//   findAll(Sort sort)            -> get all rows, sorted
//   findAll(Pageable pageable)    -> get one page of rows
public interface ProductRepository extends PagingAndSortingRepository<Product, Integer> {
}
```

Step 4 — Stage 3 (final): change to JpaRepository

JpaRepository sits at the top of the hierarchy: JpaRepository → PagingAndSortingRepository → CrudRepository. Switching to it keeps every method from the previous two stages and adds JPA-specific batch operations such as saveAll() and flush().

src/main/java/com/exam/p2/ProductRepository.java (FINAL)

```
package com.exam.p2;

import org.springframework.data.jpa.repository.JpaRepository;

/**
 * ----- STAGE 3 (FINAL) : JpaRepository -----
 * JpaRepository extends PagingAndSortingRepository, which itself extends
 * CrudRepository. So this single interface now carries ALL THREE levels:
 *
 *   From CrudRepository:            save(), findById(), deleteById() ...
 *   From PagingAndSortingRepository: findAll(Sort), findAll(Pageable)
 *   NEW in JpaRepository:            saveAll(Iterable), flush(),
 *                                    saveAndFlush(), deleteAllInBatch()
 *
 * In a real project you simply extend JpaRepository directly (as below);
 * Stages 1 and 2 above are shown only so you can see what becomes newly
 * available at each level of the hierarchy, as the practical asks.
 */
public interface ProductRepository extends JpaRepository<Product, Integer> {
}
```

Step 5 & 6 — Build ProductRestController exercising every stage

The controller below is written against the final JpaRepository version, but includes a dedicated /sorted endpoint (Stage 2 capability) and a /batch endpoint using saveAll()+flush() (Stage 3 capability), so all three hierarchy levels are demonstrably exercised.

src/main/java/com/exam/p2/ProductRestController.java

```
package com.exam.p2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    private ProductRepository productRepository;

    // ---- CREATE : POST /api/products ----
    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productRepository.save(product);
    }

    // ---- READ ALL : GET /api/products ----
    @GetMapping
    public List<Product> getAllProducts() {
        return productRepository.findAll(); // JpaRepository.findAll() returns List directly
    }

    // ---- READ ALL, SORTED : GET /api/products/sorted ----
    // Demonstrates the method that PagingAndSortingRepository introduced.
    @GetMapping("/sorted")
```

```
    public List<Product> getProductsSortedByPrice() {
        return productRepository.findAll(Sort.by("price").ascending());
    }

    // ---- BATCH INSERT : POST /api/products/batch ----
    // Demonstrates saveAll() + flush(), which are new in JpaRepository.
    @PostMapping("/batch")
    public List<Product> createProductsBatch(@RequestBody List<Product> products) {
        List<Product> saved = productRepository.saveAll(products);
        productRepository.flush(); // force Hibernate to write the batch to the DB immediately
        return saved;
    }

    // ---- UPDATE : PUT /api/products/{id} ----
    @PutMapping("/{id}")
    public Product updateProduct(@PathVariable int id, @RequestBody Product updated) {
        Optional<Product> existingOpt = productRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Product existing = existingOpt.get();
        existing.setProductName(updated.getProductName());
        existing.setCategory(updated.getCategory());
        existing.setPrice(updated.getPrice());
        existing.setQuantity(updated.getQuantity());
        return productRepository.save(existing);
    }

    // ---- DELETE : DELETE /api/products/{id} ----
    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable int id) {
        productRepository.deleteById(id);
        return "Product deleted successfully";
    }
}
```

Step 7 — Test each version's endpoints in Postman before moving on

1. With Stage 1 code, test POST /api/products and GET /api/products only (sorting/paging endpoints would not compile yet).

2. Switch to Stage 2, add the /sorted endpoint, and re-test.

3. Switch to Stage 3 (final), add /batch, and run the complete Postman collection below.

Postman API Testing

Practical 2 — Postman API Testing (Repository Hierarchy)

▸ 1. POST — create a product

POST `http://localhost:8080/api/products` **Send**

Body > raw (JSON)

```
{
  "productName": "Laptop",
  "category": "Electronics",
  "price": 55000.0,
  "quantity": 10
}
```

Response — Status: 200 OK

```
{
  "productId": 1,
  "productName": "Laptop",
  "category": "Electronics",
  "price": 55000.0,
  "quantity": 10
}
```

▸ 2. GET — sorted by price (PagingAndSortingRepository)

GET `http://localhost:8080/api/products/sorted` **Send**

Response — Status: 200 OK

```
[
  {
    "productId": 2,
    "productName": "Mouse",
    "category": "Electronics",
    "price": 499.0,
    "quantity": 50
  },
  {
    "productId": 1,
    "productName": "Laptop",
    "category": "Electronics",
    "price": 55000.0,
    "quantity": 10
  }
]
```

▸ 3. PUT — update a product

PUT `http://localhost:8080/api/products/1` **Send**

Body > raw (JSON)

```
{
  "productName": "Laptop",
  "category": "Electronics",
  "price": 52999.0,
  "quantity": 8
}
```

Response — Status: 200 OK

```
{
  "productId": 1,
  "productName": "Laptop",
  "category": "Electronics",
  "price": 52999.0,
  "quantity": 8
}
```

▸ 4. DELETE — remove a product

DELETE `http://localhost:8080/api/products/2` **Send**

Response — Status: 200 OK

```
{
  "message": "Product deleted successfully"
}
```

> ***i*** *Because Hibernate auto-creates the table, switching the repository's extends clause between stages requires no manual database changes — just restart the app.*

Practical No. 3 — Custom Query Methods: findBy, countBy and @Query (JPQL)

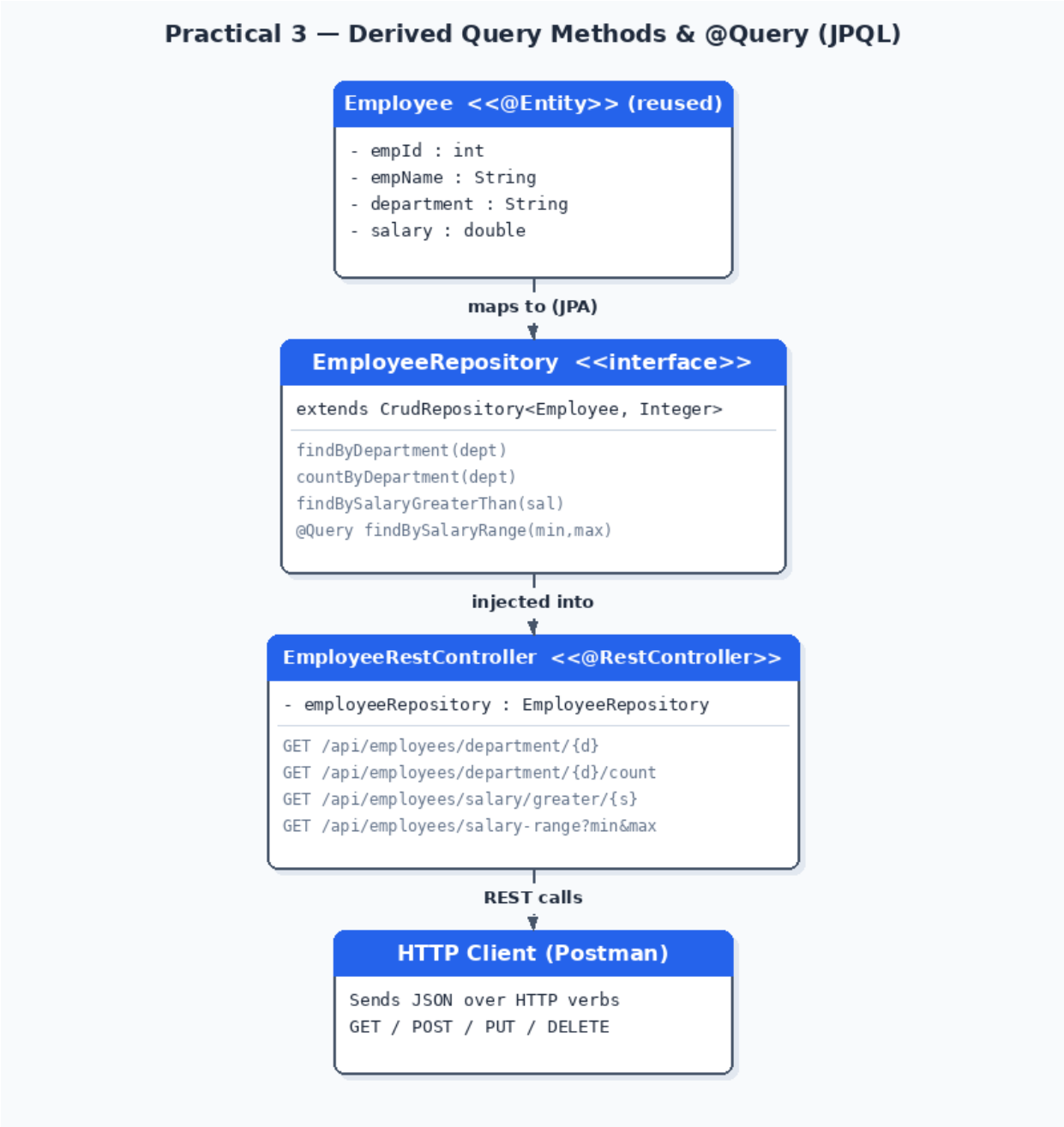
Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries using @Query for requirements that cannot be expressed by naming conventions alone.

> ***i*** *This practical reuses the Employee entity and EmployeeRepository created in Practical 1 (package com.exam.p1) — we only ADD new method declarations and new controller endpoints; nothing from Practical 1 is deleted.*

Design — Class Diagram



Step-by-Step Implementation

Step 1 & 2 — Add derived query methods to EmployeeRepository

Spring Data JPA parses a repository method's NAME at startup and builds the SQL for you — no implementation is written by hand. The keyword after "findBy"/"countBy" must match an entity field name exactly (case-sensitive to the property name).

- findByDepartment(String) → WHERE department = ?
- countByDepartment(String) → SELECT COUNT(*) ... WHERE department = ?
- findBySalaryGreaterThan(double) → WHERE salary > ?

Step 3 — Write a custom JPQL query with @Query

Some conditions (like a BETWEEN range with two bound parameters) are awkward to express as a method name, so we write the JPQL directly and bind parameters with @Param.

src/main/java/com/exam/p1/EmployeeRepository.java (extended)

```
package com.exam.p1;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;

import java.util.List;

/**
 * EmployeeRepository - extended with custom query methods.
 * (This is the SAME Employee / EmployeeRepository created in Practical 1 -
 *  we simply add more method declarations to the interface.)
 */
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {

    // ---- DERIVED QUERY : Spring parses the method name and builds the SQL ----
    // "findBy" + "Department"  ->  SELECT * FROM employee WHERE department = ?
    List<Employee> findByDepartment(String department);

    // "countBy" + "Department" ->  SELECT COUNT(*) FROM employee WHERE department = ?
    long countByDepartment(String department);

    // "findBy" + "Salary" + "GreaterThan" -> SELECT * FROM employee WHERE salary > ?
    List<Employee> findBySalaryGreaterThan(double salary);

    // ---- CUSTOM QUERY : hand-written JPQL for logic naming conventions can't express ----
    @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
    List<Employee> findBySalaryRange(@Param("min") double min, @Param("max") double max);
}
```

Step 4 & 5 — Expose each method through the REST controller + seed data

A /seed endpoint is added purely to quickly populate a few employees across departments and salary bands for testing, so the salary-range and department queries have realistic data to return.

src/main/java/com/exam/p1/EmployeeRestController.java (extended)

```
package com.exam.p1;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
```

```
/**
 * New endpoints added to EmployeeRestController (Practical 1) to expose the
 * custom query methods declared in EmployeeRepository above.
 */
@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    // ... POST / GET / GET-by-id / PUT / DELETE from Practical 1 stay unchanged ...

    // ---- Seed sample data across departments & salary bands ----
    @PostMapping("/seed")
    public List<Employee> seedEmployees() {
        employeeRepository.save(new Employee("Rahul Sharma", "IT", 55000));
        employeeRepository.save(new Employee("Sneha Patil", "IT", 61000));
        employeeRepository.save(new Employee("Aditya Rane", "HR", 42000));
        employeeRepository.save(new Employee("Priya Deshmukh", "Finance", 70000));
        List<Employee> all = new java.util.ArrayList<>();
        employeeRepository.findAll().forEach(all::add);
        return all;
    }

    // ---- Derived query : GET /api/employees/department/{department} ----
    @GetMapping("/department/{department}")
    public List<Employee> getByDepartment(@PathVariable String department) {
        return employeeRepository.findByDepartment(department);
    }

    // ---- Derived query : GET /api/employees/department/{department}/count ----
    @GetMapping("/department/{department}/count")
    public long countByDepartment(@PathVariable String department) {
        return employeeRepository.countByDepartment(department);
    }

    // ---- Derived query : GET /api/employees/salary/greater/{salary} ----
    @GetMapping("/salary/greater/{salary}")
    public List<Employee> getBySalaryGreaterThan(@PathVariable double salary) {
        return employeeRepository.findBySalaryGreaterThan(salary);
    }

    // ---- Custom JPQL query : GET /api/employees/salary-range?min=..&max=.. ----
    @GetMapping("/salary-range")
    public List<Employee> getBySalaryRange(
            @RequestParam double min, @RequestParam double max) {
        return employeeRepository.findBySalaryRange(min, max);
    }
}
```

Step 6 — Test each query endpoint from Postman

1. Call POST /api/employees/seed once to insert sample rows.

2. Call each GET endpoint below and manually verify the returned rows actually satisfy the condition (e.g. every row from /department/IT really has department = "IT").

3. Use PUT /api/employees/{id} (from Practical 1) to change an employee's salary, then re-call /salary-range to confirm the result set changes accordingly.

4. Use DELETE /api/employees/{id} to remove one employee, then re-call /department/{d}/count and confirm the count decreased by 1.

Postman API Testing

Practical 3 — Postman API Testing (Derived & @Query Methods)

▸ 1. GET — employees by department (derived query)

GET http://localhost:8080/api/employees/department/IT Send

Response Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  },
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

▸ 2. GET — count employees in a department

GET http://localhost:8080/api/employees/department/IT/count Send

Response Status: 200 OK

```
2
```

▸ 3. GET — employees with salary greater than X

GET http://localhost:8080/api/employees/salary/greater/60000 Send

Response Status: 200 OK

```
[
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

▸ 4. GET — custom JPQL @Query for salary range

GET http://localhost:8080/api/employees/salary-range?min=50000&max=65000 Send

Response Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Rahul Sharma",
    "department": "IT",
    "salary": 55000.0
  },
  {
    "empId": 2,
    "empName": "Sneha Patil",
    "department": "IT",
    "salary": 61000.0
  }
]
```

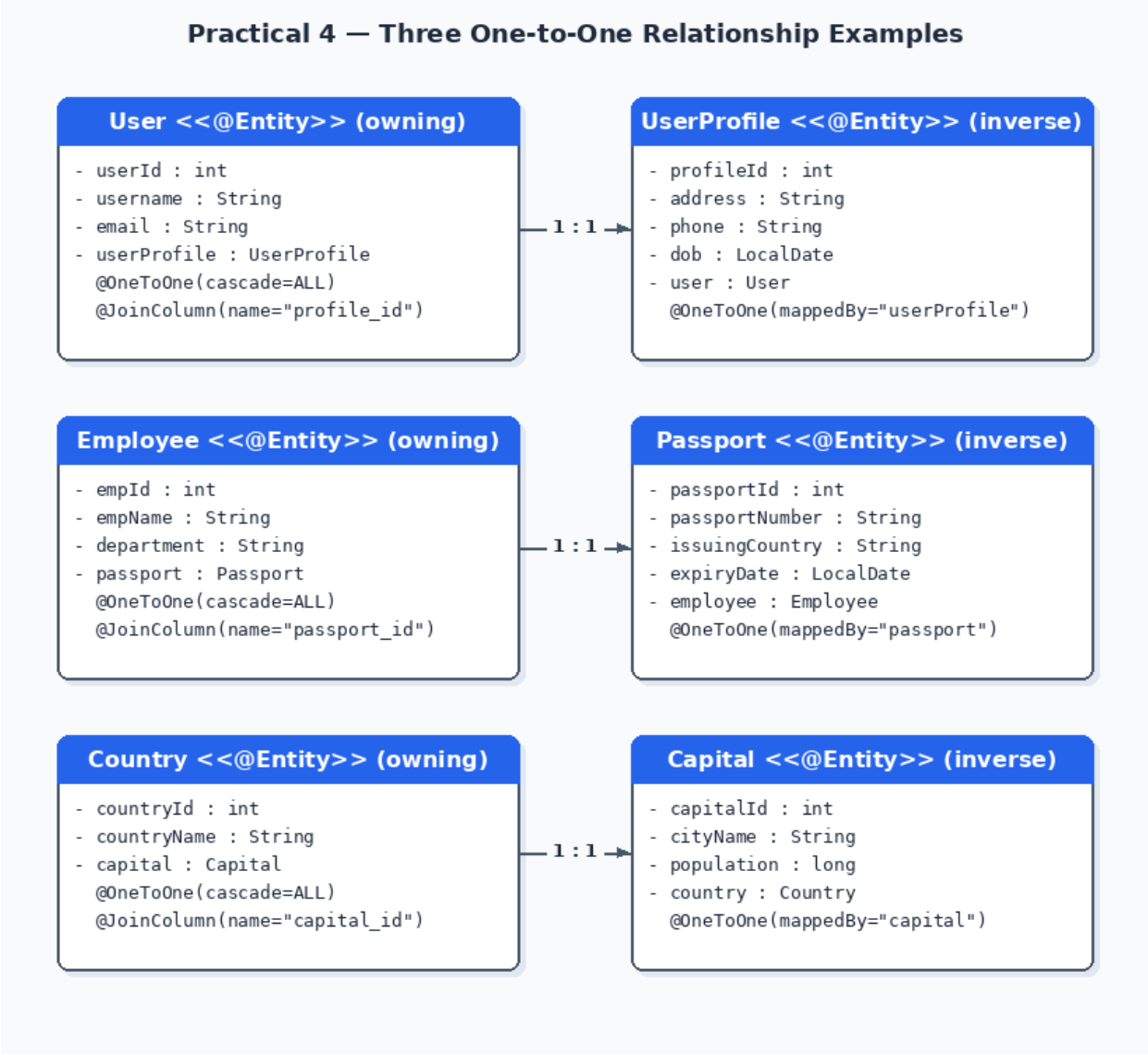
Practical No. 4 — One-to-One Mapping: Three Worked Examples

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement the One-to-One relationship using @OneToOne, @JoinColumn (owning side) and mappedBy (inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent pairs reinforce the pattern; full CRUD is built for Example 1 (User-UserProfile).

Design — Entity-Relationship Diagram (all three examples)



Key Concept: Owning Side vs Inverse Side

- The OWNING side (User, Employee, Country) carries the physical foreign-key column via @JoinColumn, and cascade = CascadeType.ALL on that side means saving/deleting the parent automatically saves/deletes the child.

- The INVERSE side (UserProfile, Passport, Capital) declares mappedBy = "<field name on owning side>" — it does NOT create a column; it only lets Java code navigate back from child to parent.

> *i* *@JsonIgnoreProperties is added on the owning side's relationship field purely to stop Jackson from serializing User -> profile -> user -> profile ... in an infinite loop when returning JSON.*

Example 1 — User ↔ UserProfile (Full CRUD)

Step 1 — UserProfile entity (inverse side)

src/main/java/com/exam/p4/UserProfile.java

```
package com.exam.p4;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int profileId;

    private String address;
    private String phone;
    private LocalDate dob;

    // Inverse side of the relationship: mappedBy points to the field name
    // ("userProfile") on the OWNING entity (User). This side does NOT own
    // the foreign key column - it just lets Java navigate profile -> user.
    @OneToOne(mappedBy = "userProfile")
    private User user;

    public UserProfile() {}

    public UserProfile(String address, String phone, LocalDate dob) {
        this.address = address;
        this.phone = phone;
        this.dob = dob;
    }

    public int getProfileId() { return profileId; }
    public void setProfileId(int profileId) { this.profileId = profileId; }
    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }
    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }
    public LocalDate getDob() { return dob; }
    public void setDob(LocalDate dob) { this.dob = dob; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}
```

Step 1 (contd.) — User entity (owning side)

src/main/java/com/exam/p4/User.java

```
package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class User {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;

    private String username;
    private String email;

    // Owning side: this entity's table holds the foreign key column
    // "profile_id" (@JoinColumn). cascade = ALL means: whenever a User is
    // saved/deleted, JPA automatically saves/deletes its UserProfile too -
    // we never need to call userProfileRepository.save() separately.
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    @JsonIgnoreProperties("user") // avoid infinite User->profile->user->... JSON loop
    private UserProfile userProfile;

    public User() {}

    public User(String username, String email, UserProfile userProfile) {
        this.username = username;
        this.email = email;
        this.userProfile = userProfile;
    }

    public int getUserId() { return userId; }
    public void setUserId(int userId) { this.userId = userId; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public UserProfile getUserProfile() { return userProfile; }
    public void setUserProfile(UserProfile userProfile) { this.userProfile = userProfile; }
}
```

Step 2 — UserRepository and UserRestController (full CRUD)

src/main/java/com/exam/p4/UserRepository.java

```
package com.exam.p4;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {
}
```

src/main/java/com/exam/p4/UserRestController.java

```
package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/users")
public class UserRestController {

    @Autowired
    private UserRepository userRepository;

    // POST /api/users - accepts nested profile JSON; cascade=ALL saves both rows
    @PostMapping
```

```
    public User createUser(@RequestBody User user) {
        if (user.getUserProfile() != null) {
            user.getUserProfile().setUser(user); // keep both sides of the link in sync
        }
        return userRepository.save(user);
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable int id) {
        return userRepository.findById(id).orElse(null);
    }

    // PUT /api/users/{id} - updates the user AND its nested profile
    @PutMapping("/{id}")
    public User updateUser(@PathVariable int id, @RequestBody User updated) {
        Optional<User> existingOpt = userRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        User existing = existingOpt.get();
        existing.setUsername(updated.getUsername());
        existing.setEmail(updated.getEmail());
        if (updated.getUserProfile() != null && existing.getUserProfile() != null) {
            existing.getUserProfile().setAddress(updated.getUserProfile().getAddress());
            existing.getUserProfile().setPhone(updated.getUserProfile().getPhone());
            existing.getUserProfile().setDob(updated.getUserProfile().getDob());
        }
        return userRepository.save(existing);
    }

    // DELETE /api/users/{id} - cascade = ALL removes the linked UserProfile too
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable int id) {
        userRepository.deleteById(id);
        return "User (and linked profile) deleted successfully";
    }
}
```

Example 2 — Employee ↔ Passport (same pattern)

Step 3: repeat the identical owning/inverse pattern under /api/employees-passport.

src/main/java/com/exam/p4/Passport.java

```
package com.exam.p4;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class Passport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int passportId;

    private String passportNumber;
    private String issuingCountry;
    private LocalDate expiryDate;

    @OneToOne(mappedBy = "passport")
    private Employee employee;
```

```
    public Passport() {}
    public Passport(String passportNumber, String issuingCountry, LocalDate expiryDate) {
        this.passportNumber = passportNumber;
        this.issuingCountry = issuingCountry;
        this.expiryDate = expiryDate;
    }
    // Getters & Setters
    public int getPassportId() { return passportId; }
    public void setPassportId(int passportId) { this.passportId = passportId; }
    public String getPassportNumber() { return passportNumber; }
    public void setPassportNumber(String passportNumber) { this.passportNumber = passportNumber; }
    public String getIssuingCountry() { return issuingCountry; }
    public void setIssuingCountry(String issuingCountry) { this.issuingCountry = issuingCountry; }
    public LocalDate getExpiryDate() { return expiryDate; }
    public void setExpiryDate(LocalDate expiryDate) { this.expiryDate = expiryDate; }
    public Employee getEmployee() { return employee; }
    public void setEmployee(Employee employee) { this.employee = employee; }
}
```

src/main/java/com/exam/p4/Employee.java

```
package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "passport_id")
    @JsonIgnoreProperties("employee")
    private Passport passport;

    public Employee() {}
    public Employee(String empName, String department, Passport passport) {
        this.empName = empName; this.department = department; this.passport = passport;
    }
    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public Passport getPassport() { return passport; }
    public void setPassport(Passport passport) { this.passport = passport; }
}
```

src/main/java/com/exam/p4/EmployeePassportController.java

```
package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

interface EmployeePassportRepository extends JpaRepository<Employee, Integer> {}
```

```
@RestController
@RequestMapping("/api/employees-passport")
class EmployeePassportController {

    @Autowired
    private EmployeePassportRepository employeeRepository;

    // POST /api/employees-passport - create employee with nested passport
    @PostMapping
    public Employee create(@RequestBody Employee employee) {
        if (employee.getPassport() != null) {
            employee.getPassport().setEmployee(employee);
        }
        return employeeRepository.save(employee);
    }

    // GET /api/employees-passport - list all, each with its passport
    @GetMapping
    public List<Employee> getAll() {
        return employeeRepository.findAll();
    }
}
```

Example 3 — Country ↔ Capital (same pattern)

Step 4: repeat the pattern once more under /api/countries.

src/main/java/com/exam/p4/Country.java

```
package com.exam.p4;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Country {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int countryId;

    private String countryName;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "capital_id")
    @JsonIgnoreProperties("country")
    private Capital capital;

    public Country() {}
    public Country(String countryName, Capital capital) {
        this.countryName = countryName; this.capital = capital;
    }
    public int getCountryId() { return countryId; }
    public void setCountryId(int countryId) { this.countryId = countryId; }
    public String getCountryName() { return countryName; }
    public void setCountryName(String countryName) { this.countryName = countryName; }
    public Capital getCapital() { return capital; }
    public void setCapital(Capital capital) { this.capital = capital; }
}
```

src/main/java/com/exam/p4/Capital.java

```
package com.exam.p4;

import jakarta.persistence.*;
```

```
@Entity
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int capitalId;

    private String cityName;
    private long population;

    @OneToOne(mappedBy = "capital")
    private Country country;

    public Capital() {}
    public Capital(String cityName, long population) {
        this.cityName = cityName; this.population = population;
    }
    public int getCapitalId() { return capitalId; }
    public void setCapitalId(int capitalId) { this.capitalId = capitalId; }
    public String getCityName() { return cityName; }
    public void setCityName(String cityName) { this.cityName = cityName; }
    public long getPopulation() { return population; }
    public void setPopulation(long population) { this.population = population; }
    public Country getCountry() { return country; }
    public void setCountry(Country country) { this.country = country; }
}
```

src/main/java/com/exam/p4/CountryController.java

```
package com.exam.p4;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

interface CountryRepository extends JpaRepository<Country, Integer> {}

@RestController
@RequestMapping("/api/countries")
class CountryController {

    @Autowired
    private CountryRepository countryRepository;

    // POST /api/countries - create country with nested capital
    @PostMapping
    public Country create(@RequestBody Country country) {
        if (country.getCapital() != null) {
            country.getCapital().setCountry(country);
        }
        return countryRepository.save(country);
    }

    // GET /api/countries - list all countries with their capitals
    @GetMapping
    public List<Country> getAll() {
        return countryRepository.findAll();
    }
}
```

Step 5 & 6 — Verify cascading and test in Postman

1. Run full CRUD (POST, GET, PUT, DELETE) against Example 1 in Postman, in that order.

2. For Examples 2 and 3, at minimum repeat POST and GET to confirm the same cascading pattern generalizes.

3. After a DELETE on /api/users/{id}, check the H2 console: the linked row in USER_PROFILE should also be gone, proving cascade = ALL removed the child too.

Postman API Testing

Practical 4 — Postman API Testing (One-to-One x3)

▸ 1. POST — create User with nested UserProfile (cascade save)

POST http://localhost:8080/api/users Send

Body > raw (JSON)

```
{
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

Response Status: 200 OK

```
{
  "userId": 1,
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "profileId": 1,
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

▸ 2. GET — fetch user with profile

GET http://localhost:8080/api/users/1 Send

Response Status: 200 OK

```
{
  "userId": 1,
  "username": "amit01",
  "email": "amit01@example.com",
  "userProfile": {
    "profileId": 1,
    "address": "Pune, MH",
    "phone": "9876543210",
    "dob": "1998-05-14"
  }
}
```

▸ 3. POST — Example 2: Employee with nested Passport

POST http://localhost:8080/api/employees-passport Send

Body > raw (JSON)

```
{
  "empName": "Kiran Rao",
  "department": "HR",
  "passport": {
    "passportNumber": "P1234567",
    "issuingCountry": "India",
    "expiryDate": "2030-01-01"
  }
}
```

Response Status: 200 OK

```
{
  "empId": 1,
  "empName": "Kiran Rao",
  "department": "HR",
  "passport": {
    "passportId": 1,
    "passportNumber": "P1234567",
    "issuingCountry": "India",
    "expiryDate": "2030-01-01"
  }
}
```

▸ 4. POST — Example 3: Country with nested Capital

POST http://localhost:8080/api/countries Send

Body > raw (JSON)

```
{
  "countryName": "India",
  "capital": {
    "cityName": "New Delhi",
    "population": 32900000
  }
}
```

Response Status: 200 OK

```
{
  "countryId": 1,
  "countryName": "India",
  "capital": {
    "capitalId": 1,
    "cityName": "New Delhi",
    "population": 32900000
  }
}
```

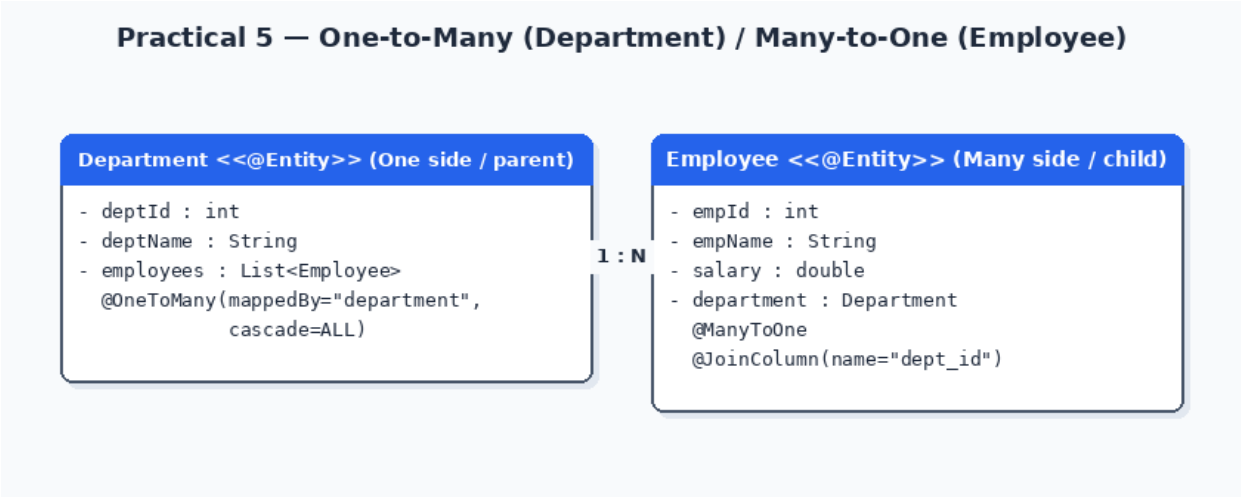
Practical No. 5 — One-to-Many / Many-to-One Mapping: Department and Employee

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee → Department) relationship using mappedBy on the parent side and @JoinColumn on the child side, and to persist a parent with a list of children in a single request.

Design — Entity-Relationship Diagram



Step-by-Step Implementation

Step 1 — Employee entity (the "Many" side)

Employee owns the foreign key: its table gets a dept_id column pointing back to Department.

src/main/java/com/exam/p5/Employee.java

```
package com.exam.p5;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private double salary;

    // MANY side: this table stores the foreign key column "dept_id".
    // Many Employee rows can point to the same Department.
    @ManyToOne
    @JoinColumn(name = "dept_id")
```

```
    @JsonIgnoreProperties("employees") // stop Department -> employees -> department -> ... loop
    private Department department;

    public Employee() {}
    public Employee(String empName, double salary) {
        this.empName = empName;
        this.salary = salary;
    }

    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public Department getDepartment() { return department; }
    public void setDepartment(Department department) { this.department = department; }
}
```

Step 2 — Department entity (the "One" side)

mappedBy = "department" tells Hibernate the relationship is already described on the Employee side — Department does not get its own foreign-key column. orphanRemoval = true additionally allows deleting a child by simply removing it from this list.

src/main/java/com/exam/p5/Department.java

```
package com.exam.p5;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int deptId;

    private String deptName;

    // ONE side: mappedBy = "department" points at the field name on the
    // MANY (Employee) side. cascade = ALL means saving a Department with a
    // populated employees list also inserts every Employee row.
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Employee> employees;

    public Department() {}
    public Department(String deptName, List<Employee> employees) {
        this.deptName = deptName;
        this.employees = employees;
    }

    public int getDeptId() { return deptId; }
    public void setDeptId(int deptId) { this.deptId = deptId; }
    public String getDeptName() { return deptName; }
    public void setDeptName(String deptName) { this.deptName = deptName; }
    public List<Employee> getEmployees() { return employees; }
    public void setEmployees(List<Employee> employees) { this.employees = employees; }
}
```

Step 3 — DepartmentRepository

src/main/java/com/exam/p5/DepartmentRepository.java

```
package com.exam.p5;

import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository extends JpaRepository<Department, Integer> {
}
```

Step 4-7 — DepartmentRestController (full CRUD, both directions)

Notice the important line in createDepartment(): each child Employee's setDepartment(department) must be called before saving, or the dept_id foreign key would be written as NULL — cascade only tells JPA to also INSERT the child rows, it does not set the back-reference for you.

src/main/java/com/exam/p5/DepartmentRestController.java

```
package com.exam.p5;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/departments")
public class DepartmentRestController {

    @Autowired
    private DepartmentRepository departmentRepository;

    // ---- CREATE : POST /api/departments ----
    // Accepts a department name + list of employee objects, saves the whole
    // graph in one call thanks to cascade = ALL on Department.employees.
    @PostMapping
    public Department createDepartment(@RequestBody Department department) {
        if (department.getEmployees() != null) {
            // IMPORTANT: set the back-reference on every child before saving,
            // otherwise the dept_id foreign-key column would be left NULL.
            department.getEmployees().forEach(emp -> emp.setDepartment(department));
        }
        return departmentRepository.save(department);
    }

    // ---- READ : GET /api/departments/{id} ----
    @GetMapping("/{id}")
    public Department getDepartment(@PathVariable int id) {
        return departmentRepository.findById(id).orElse(null);
    }

    // ---- READ only the employee list : GET /api/departments/{id}/employees ----
    @GetMapping("/{id}/employees")
    public List<Employee> getEmployeesOfDepartment(@PathVariable int id) {
        Optional<Department> dept = departmentRepository.findById(id);
        return dept.map(Department::getEmployees).orElse(null);
    }

    // ---- RENAME department : PUT /api/departments/{id} ----
    @PutMapping("/{id}")
    public Department renameDepartment(@PathVariable int id, @RequestBody Department updated) {
        Optional<Department> existingOpt = departmentRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Department existing = existingOpt.get();
        existing.setDeptName(updated.getDeptName());
        return departmentRepository.save(existing);
    }
```

```
    // ---- UPDATE employee inside a department : PUT .../{deptId}/employees/{empId} ----
    // Demonstrates Many-to-One navigation from Employee back to Department.
    @PutMapping("/{deptId}/employees/{empId}")
    public Employee updateEmployeeInDepartment(
            @PathVariable int deptId, @PathVariable int empId, @RequestBody Employee updated) {
        Optional<Department> deptOpt = departmentRepository.findById(deptId);
        if (deptOpt.isEmpty()) return null;
        Department department = deptOpt.get();
        for (Employee emp : department.getEmployees()) {
            if (emp.getEmpId() == empId) {
                emp.setEmpName(updated.getEmpName());
                emp.setSalary(updated.getSalary());
                departmentRepository.save(department); // cascades change to Employee row
                return emp;
            }
        }
        return null;
    }

    // ---- DELETE one employee from a department : DELETE .../{deptId}/employees/{empId} ----
    @DeleteMapping("/{deptId}/employees/{empId}")
    public String removeEmployeeFromDepartment(
            @PathVariable int deptId, @PathVariable int empId) {
        Optional<Department> deptOpt = departmentRepository.findById(deptId);
        if (deptOpt.isEmpty()) return "Department not found";
        Department department = deptOpt.get();
        // orphanRemoval = true on the @OneToMany means: removing an Employee
        // from this list and saving the parent DELETES that Employee's row.
        department.getEmployees().removeIf(emp -> emp.getEmpId() == empId);
        departmentRepository.save(department);
        return "Employee removed from department";
    }

    // ---- DELETE a whole department (+ all its employees, via cascade) ----
    @DeleteMapping("/{id}")
    public String deleteDepartment(@PathVariable int id) {
        departmentRepository.deleteById(id);
        return "Department and all its employees deleted successfully";
    }
}
```

Step 8 — Test in Postman

1. POST a department with a nested list of employees — confirm one call inserts both the department row and every employee row.

2. GET /api/departments/{id} and GET /api/departments/{id}/employees to confirm both read endpoints.

3. PUT /api/departments/{deptId}/employees/{empId} to update a single employee's salary from within the department.

4. DELETE one employee, then DELETE the whole department, and confirm via the H2 console that cascading removed every related row.

Postman API Testing

Practical 5 — Postman API Testing (One-to-Many / Many-to-One)

▸ 1. POST — create Department with a list of Employees (cascade)

POST `http://localhost:8080/api/departments` — Send

Body > raw (JSON)

```
{
  "deptName": "Computer Science",
  "employees": [
    {
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

Response — Status: 200 OK

```
{
  "deptId": 1,
  "deptName": "Computer Science",
  "employees": [
    {
      "empId": 1,
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empId": 2,
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

▸ 2. GET — department with employees

GET `http://localhost:8080/api/departments/1` — Send

Response — Status: 200 OK

```
{
  "deptId": 1,
  "deptName": "Computer Science",
  "employees": [
    {
      "empId": 1,
      "empName": "Neha Joshi",
      "salary": 48000.0
    },
    {
      "empId": 2,
      "empName": "Vikas Nair",
      "salary": 52000.0
    }
  ]
}
```

▸ 3. PUT — update one employee within a department

PUT `http://localhost:8080/api/departments/1/employees/1` — Send

Body > raw (JSON)

```
{
  "empName": "Neha Joshi",
  "salary": 51000.0
}
```

Response — Status: 200 OK

```
{
  "empId": 1,
  "empName": "Neha Joshi",
  "salary": 51000.0,
  "department": {
    "deptId": 1,
    "deptName": "Computer Science"
  }
}
```

▸ 4. DELETE — remove one employee from a department

DELETE `http://localhost:8080/api/departments/1/employees/2` — Send

Response — Status: 200 OK

```
{
  "message": "Employee removed from department"
}
```

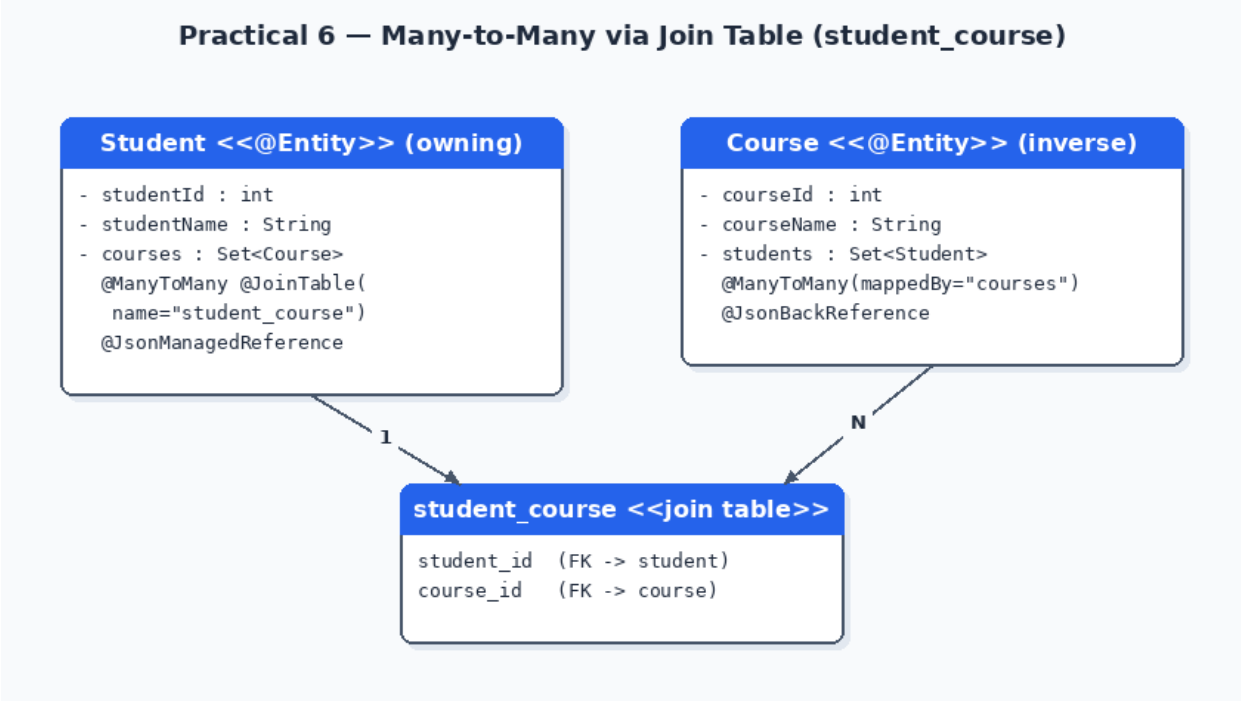
Practical No. 6 — Many-to-Many Mapping: Student and Course

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement a Many-to-Many relationship between Student and Course using @ManyToMany and a join table via @JoinTable, and to avoid infinite JSON recursion using @JsonManagedReference / @JsonBackReference.

Design — Entity-Relationship Diagram



> ***i*** *Unlike @OneToOne / @OneToMany, a Many-to-Many relationship cannot be stored with a single foreign-key column on either table — JPA needs a separate JOIN TABLE (student_course) holding one row per (student, course) pairing.*

Step-by-Step Implementation

Step 1 — Create Course (inverse side)

src/main/java/com/exam/p6/Course.java

```
package com.exam.p6;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;
import java.util.Set;

@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int courseId;

    private String courseName;
```

```
    // Inverse side: mappedBy points to the field name ("courses") on Student.
    // @JsonBackReference stops Jackson recursing back into Student -> courses -> ...
    @ManyToMany(mappedBy = "courses")
    @JsonBackReference
    private Set<Student> students;

    public Course() {}
    public Course(String courseName) { this.courseName = courseName; }

    public int getCourseId() { return courseId; }
    public void setCourseId(int courseId) { this.courseId = courseId; }
    public String getCourseName() { return courseName; }
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public Set<Student> getStudents() { return students; }
    public void setStudents(Set<Student> students) { this.students = students; }
}
```

Step 2 & 3 — Create Student (owning side) with @JoinTable

@JoinTable is declared only on the owning side. joinColumns refers to THIS entity (Student); inverseJoinColumns refers to the OTHER entity (Course).

src/main/java/com/exam/p6/Student.java

```
package com.exam.p6;

import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;
import java.util.Set;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int studentId;

    private String studentName;

    // Owning side: @JoinTable physically creates the student_course table with
    // two foreign-key columns. @JsonManagedReference is the "forward" half of
    // the pair that lets Jackson serialize this collection safely.
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    @JsonManagedReference
    private Set<Course> courses;

    public Student() {}
    public Student(String studentName, Set<Course> courses) {
        this.studentName = studentName;
        this.courses = courses;
    }

    public int getStudentId() { return studentId; }
    public void setStudentId(int studentId) { this.studentId = studentId; }
    public String getStudentName() { return studentName; }
    public void setStudentName(String studentName) { this.studentName = studentName; }
    public Set<Course> getCourses() { return courses; }
    public void setCourses(Set<Course> courses) { this.courses = courses; }
}
```

Step 4 — Repositories

src/main/java/com/exam/p6/StudentRepository.java

```
package com.exam.p6;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Integer> {
}
```

src/main/java/com/exam/p6/CourseRepository.java

```
package com.exam.p6;

import org.springframework.data.jpa.repository.JpaRepository;

public interface CourseRepository extends JpaRepository<Course, Integer> {
}
```

Step 5, 6 & 7 — REST controller with full CRUD + enroll/unenroll

Notice createStudent() and replaceStudentCourses() re-fetch each Course by id rather than trusting the incoming JSON object as-is — this prevents Hibernate from trying to INSERT a brand-new duplicate Course row for a course that already exists.

src/main/java/com/exam/p6/StudentCourseRestController.java

```
package com.exam.p6;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.HashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;

@RestController
public class StudentCourseRestController {

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private CourseRepository courseRepository;

    // ---- CREATE : POST /api/students ----
    // Accepts a student with a set of Course objects that ALREADY EXIST
    // (only their courseId needs to be supplied); we look each one up so
    // Hibernate reuses the existing row instead of trying to re-insert it.
    @PostMapping("/api/students")
    public Student createStudent(@RequestBody Student student) {
        Set<Course> resolved = new HashSet<>();
        for (Course c : student.getCourses()) {
            courseRepository.findById(c.getCourseId()).ifPresent(resolved::add);
        }
        student.setCourses(resolved);
        return studentRepository.save(student);
    }

    // ---- ENROLL into one more course : POST /api/students/{id}/enroll/{courseId} ----
    @PostMapping("/api/students/{id}/enroll/{courseId}")
    public Student enroll(@PathVariable int id, @PathVariable int courseId) {
        Optional<Student> studentOpt = studentRepository.findById(id);
        Optional<Course> courseOpt = courseRepository.findById(courseId);
```

```
        if (studentOpt.isEmpty() || courseOpt.isEmpty()) return null;
        Student student = studentOpt.get();
        student.getCourses().add(courseOpt.get());
        return studentRepository.save(student);
    }

    // ---- READ : GET /api/students/{id} ----
    @GetMapping("/api/students/{id}")
    public Student getStudent(@PathVariable int id) {
        return studentRepository.findById(id).orElse(null);
    }

    // ---- READ (inverse side) : GET /api/courses/{id} ----
    @GetMapping("/api/courses/{id}")
    public Course getCourse(@PathVariable int id) {
        return courseRepository.findById(id).orElse(null);
    }

    // ---- UPDATE course details : PUT /api/courses/{id} ----
    @PutMapping("/api/courses/{id}")
    public Course updateCourse(@PathVariable int id, @RequestBody Course updated) {
        Optional<Course> existingOpt = courseRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Course existing = existingOpt.get();
        existing.setCourseName(updated.getCourseName());
        return courseRepository.save(existing);
    }

    // ---- REPLACE a student's entire course set : PUT /api/students/{id} ----
    @PutMapping("/api/students/{id}")
    public Student replaceStudentCourses(@PathVariable int id, @RequestBody Student updated) {
        Optional<Student> existingOpt = studentRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Student existing = existingOpt.get();
        existing.setStudentName(updated.getStudentName());
        Set<Course> resolved = new HashSet<>();
        for (Course c : updated.getCourses()) {
            courseRepository.findById(c.getCourseId()).ifPresent(resolved::add);
        }
        existing.setCourses(resolved);
        return studentRepository.save(existing);
    }

    // ---- UNENROLL : DELETE /api/students/{studentId}/unenroll/{courseId} ----
    // Removes only the join-table row - neither Student nor Course is deleted.
    @DeleteMapping("/api/students/{studentId}/unenroll/{courseId}")
    public String unenroll(@PathVariable int studentId, @PathVariable int courseId) {
        Optional<Student> studentOpt = studentRepository.findById(studentId);
        if (studentOpt.isEmpty()) return "Student not found";
        Student student = studentOpt.get();
        student.getCourses().removeIf(c -> c.getCourseId() == courseId);
        studentRepository.save(student);
        return "Student unenrolled from course";
    }

    // ---- DELETE a course entirely : DELETE /api/courses/{id} ----
    @DeleteMapping("/api/courses/{id}")
    public String deleteCourse(@PathVariable int id) {
        List<Student> allStudents = studentRepository.findAll();
        for (Student s : allStudents) {
            s.getCourses().removeIf(c -> c.getCourseId() == id); // clear join rows first
            studentRepository.save(s);
        }
        courseRepository.deleteById(id);
        return "Course deleted successfully";
    }
}
```

Step 8 — Test in Postman

1. First create at least 3 courses (POST /api/courses, if you add that convenience endpoint, or insert directly) so their ids exist before creating a student.

2. Create a student enrolled in two courses, then call the enroll endpoint to add a third.

3. View the same relationship from the Course side to confirm bidirectionality.

4. Unenroll one course and delete another entirely, verifying the join-table rows update correctly in your database (H2 console or MySQL Workbench).

Postman API Testing

Practical 6 — Postman API Testing (Many-to-Many)

▸ 1. POST — create a student enrolled in two existing courses

POST `http://localhost:8080/api/students` — Send

Body > raw (JSON)

```
{
  "studentName": "Ananya Kulkarni",
  "courses": [
    {
      "courseId": 1
    },
    {
      "courseId": 2
    }
  ]
}
```

Response — Status: 200 OK

```
{
  "studentId": 1,
  "studentName": "Ananya Kulkarni",
  "courses": [
    {
      "courseId": 1,
      "courseName": "DBMS"
    },
    {
      "courseId": 2,
      "courseName": "Java"
    }
  ]
}
```

▸ 2. POST — enroll an existing student into one more course

POST `http://localhost:8080/api/students/1/enroll/3` — Send

Response — Status: 200 OK

```
{
  "studentId": 1,
  "studentName": "Ananya Kulkarni",
  "courses": [
    {
      "courseId": 1,
      "courseName": "DBMS"
    },
    {
      "courseId": 2,
      "courseName": "Java"
    },
    {
      "courseId": 3,
      "courseName": "Spring Boot"
    }
  ]
}
```

▸ 3. GET — view a course and its enrolled students

GET `http://localhost:8080/api/courses/2` — Send

Response — Status: 200 OK

```
{
  "courseId": 2,
  "courseName": "Java",
  "students": [
    {
      "studentId": 1,
      "studentName": "Ananya Kulkarni"
    }
  ]
}
```

▸ 4. DELETE — unenroll a student from one course (join row only)

DELETE `http://localhost:8080/api/students/1/unenroll/3` — Send

Response — Status: 200 OK

```
{
  "message": "Student unenrolled from course"
}
```

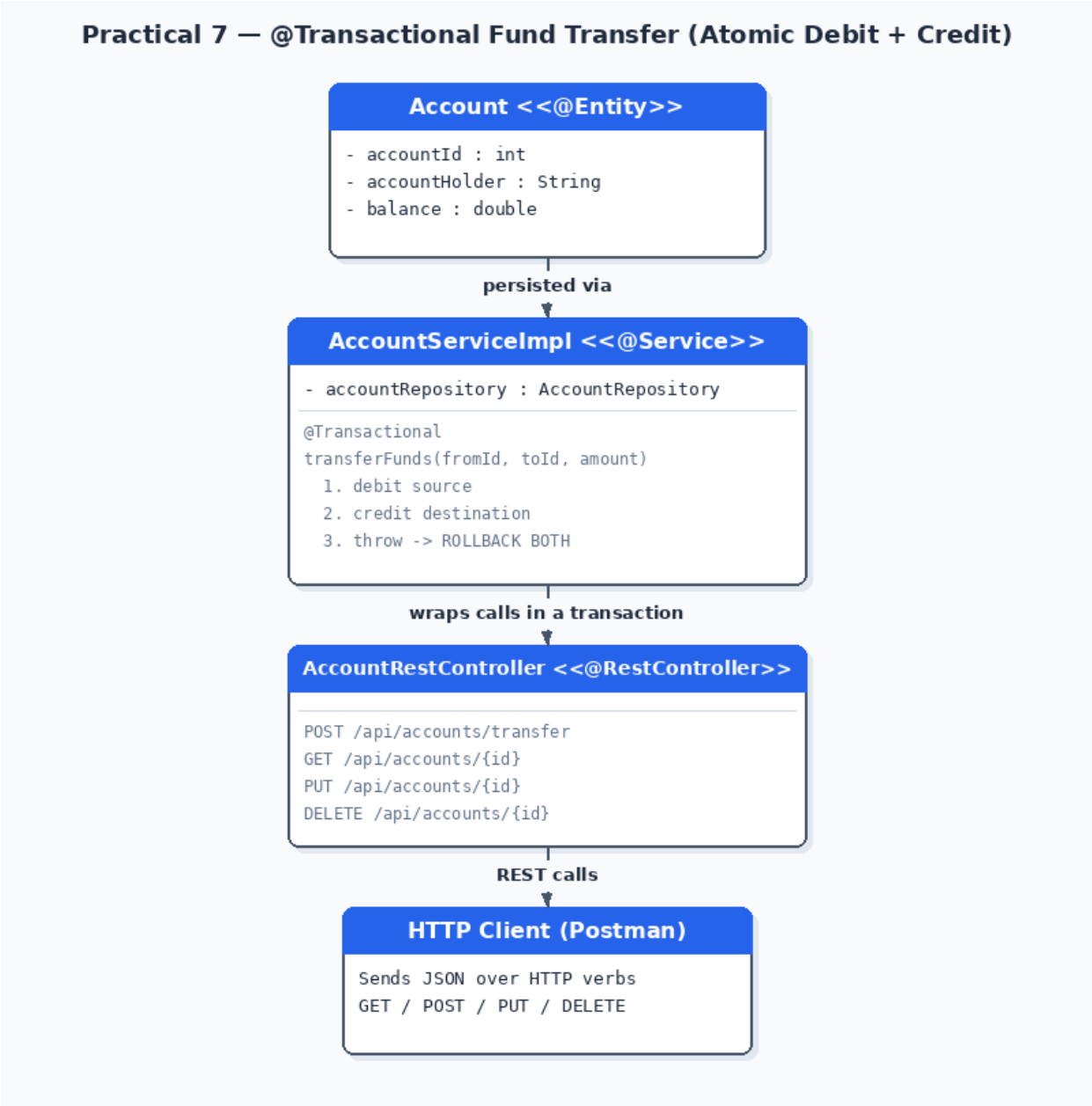
Practical No. 7 — Transaction Management with @Transactional: Bank Fund Transfer

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To understand declarative transaction management in Spring Data JPA, and to ensure atomicity across multiple repository operations (debit + credit) using @Transactional, including automatic rollback when a runtime exception occurs.

Design — Service Flow Diagram



> *i Without @Transactional, if the credit step failed after the debit step had already been saved, the source account would lose money that never reached the destination. @Transactional guarantees both steps succeed together, or neither does.*

Step-by-Step Implementation

Step 1 — Account entity and repository

src/main/java/com/exam/p7/Account.java

```
package com.exam.p7;

import jakarta.persistence.*;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int accountId;

    private String accountHolder;
    private double balance;

    public Account() {}
    public Account(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    public int getAccountId() { return accountId; }
    public void setAccountId(int accountId) { this.accountId = accountId; }
    public String getAccountHolder() { return accountHolder; }
    public void setAccountHolder(String accountHolder) { this.accountHolder = accountHolder; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
}
```

src/main/java/com/exam/p7/AccountRepository.java

```
package com.exam.p7;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountRepository extends JpaRepository<Account, Integer> {
}
```

Step 2 — Custom unchecked exception

src/main/java/com/exam/p7/InsufficientBalanceException.java

```
package com.exam.p7;

// A RuntimeException (unchecked) - Spring's default @Transactional rollback
// rule only rolls back for RuntimeExceptions, not checked Exceptions, so we
// deliberately extend RuntimeException here.
public class InsufficientBalanceException extends RuntimeException {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

Step 3 & 4 — AccountService with @Transactional transferFunds()

src/main/java/com/exam/p7/AccountService.java

```
package com.exam.p7;

public interface AccountService {
    Account transferFunds(int fromId, int toId, double amount);
    Account getAccount(int id);
    Account updateHolderName(int id, String newName);
    void closeAccount(int id);
}
```

src/main/java/com/exam/p7/AccountServiceImpl.java

```
package com.exam.p7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Optional;

@Service
public class AccountServiceImpl implements AccountService {

    @Autowired
    private AccountRepository accountRepository;

    /**
     * @Transactional wraps the ENTIRE method body in a single database
     * transaction. If ANY RuntimeException is thrown anywhere inside this
     * method - even after the debit has already been save()'d - Spring
     * rolls back every change made during the method, as if none of it
     * ever happened. This is what makes fund transfers atomic.
     */
    @Override
    @Transactional
    public Account transferFunds(int fromId, int toId, double amount) {
        // 1) Fetch source account
        Account from = accountRepository.findById(fromId)
                .orElseThrow(() -> new InsufficientBalanceException("Source account not found"));

        // 2) Check balance BEFORE debiting
        if (from.getBalance() < amount) {
            throw new InsufficientBalanceException("Insufficient balance in account " + fromId);
        }

        // 3) Debit source
        from.setBalance(from.getBalance() - amount);
        accountRepository.save(from);

        // 4) Fetch destination account
        Account to = accountRepository.findById(toId)
                .orElseThrow(() -> new InsufficientBalanceException("Destination account not found"));

        // 5) Credit destination
        to.setBalance(to.getBalance() + amount);
        accountRepository.save(to);

        // If step 4 or 5 had thrown an exception, step 3's debit would ALSO
        // be rolled back automatically because everything is inside one
        // @Transactional boundary.
        return from;
    }

    @Override
    public Account getAccount(int id) {
        return accountRepository.findById(id).orElse(null);
    }
```

```
    @Override
    public Account updateHolderName(int id, String newName) {
        Optional<Account> existingOpt = accountRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Account existing = existingOpt.get();
        existing.setAccountHolder(newName);
        return accountRepository.save(existing);
    }

    @Override
    public void closeAccount(int id) {
        Account account = accountRepository.findById(id)
                .orElseThrow(() -> new InsufficientBalanceException("Account not found"));
        if (account.getBalance() != 0) {
            throw new InsufficientBalanceException("Cannot close account with a non-zero balance");
        }
        accountRepository.deleteById(id);
    }
}
```

Step 5, 6 & 7 — AccountRestController

src/main/java/com/exam/p7/AccountRestController.java

```
package com.exam.p7;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/accounts")
public class AccountRestController {

    @Autowired
    private AccountService accountService;

    // ---- TRANSFER : POST /api/accounts/transfer ----
    @PostMapping("/transfer")
    public ResponseEntity<?> transfer(@RequestBody Map<String, Object> body) {
        try {
            int fromId = (int) body.get("fromId");
            int toId = (int) body.get("toId");
            double amount = ((Number) body.get("amount")).doubleValue();
            accountService.transferFunds(fromId, toId, amount);
            Account from = accountService.getAccount(fromId);
            Account to = accountService.getAccount(toId);
            return ResponseEntity.ok(Map.of(
                    "message", "Transfer successful",
                    "fromBalance", from.getBalance(),
                    "toBalance", to.getBalance()));
        } catch (InsufficientBalanceException ex) {
            return ResponseEntity.badRequest().body(Map.of(
                    "error", "InsufficientBalanceException", "message", ex.getMessage()));
        }
    }

    // ---- READ : GET /api/accounts/{id} ----
    @GetMapping("/{id}")
    public Account getAccount(@PathVariable int id) {
        return accountService.getAccount(id);
    }
```

```
    // ---- UPDATE (non-transactional field) : PUT /api/accounts/{id} ----
    @PutMapping("/{id}")
    public Account updateHolder(@PathVariable int id, @RequestBody Map<String, String> body) {
        return accountService.updateHolderName(id, body.get("accountHolder"));
    }

    // ---- CLOSE : DELETE /api/accounts/{id} ----
    @DeleteMapping("/{id}")
    public ResponseEntity<?> closeAccount(@PathVariable int id) {
        try {
            accountService.closeAccount(id);
            return ResponseEntity.ok(Map.of("message", "Account closed successfully"));
        } catch (InsufficientBalanceException ex) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("message", ex.getMessage()));
        }
    }
}
```

Step 8 — Verify rollback behaviour

1. Create two accounts with known balances (e.g. 20000 and 20000).

2. Transfer a small valid amount and confirm both balances update correctly.

3. Attempt to transfer an amount larger than the source balance — the request should fail with a 400 and the InsufficientBalanceException message.

4. Immediately GET both accounts again and confirm their balances are UNCHANGED from before the failed attempt — this proves @Transactional rolled back everything.

Postman API Testing

Practical 7 — Postman API Testing (@Transactional Rollback)

▸ 1. POST — successful transfer (sufficient balance)

POST `http://localhost:8080/api/accounts/transfer` **Send**

Body > raw (JSON)

```
{
  "fromId": 1,
  "toId": 2,
  "amount": 5000.0
}
```

Response — Status: 200 OK

```
{
  "message": "Transfer successful",
  "fromBalance": 15000.0,
  "toBalance": 25000.0
}
```

▸ 2. POST — failing transfer (amount > balance) → both accounts roll back

POST `http://localhost:8080/api/accounts/transfer` **Send**

Body > raw (JSON)

```
{
  "fromId": 1,
  "toId": 2,
  "amount": 999999.0
}
```

Response — Status: 400 Bad Request

```
{
  "error": "InsufficientBalanceException",
  "message": "Insufficient balance in account 1"
}
```

▸ 3. GET — confirm balances unchanged after the failed transfer

GET `http://localhost:8080/api/accounts/1` **Send**

Response — Status: 200 OK

```
{
  "accountId": 1,
  "accountHolder": "Rahul Sharma",
  "balance": 15000.0
}
```

Practical No. 8 — Pagination and Sorting with Spring Data JPA

Textbook Reference: Chapter 5 — Working with Spring Data JPA

Objective

To implement server-side pagination and sorting for a large result set using Spring Data's Pageable/PageRequest abstraction (a PagingAndSortingRepository capability), exposed through query parameters on a REST endpoint.

> ***i*** *This practical extends the Product entity and ProductRepository already built in Practical 2 (package com.exam.p2) — only one new controller method is added.*

Design — Request Flow



Step-by-Step Implementation

Step 1 — Seed sample data

Insert at least 15-20 Product rows (via repeated POST /api/products calls, or a quick batch loop) so pagination is meaningful — with only 2-3 rows every page would look the same.

Step 2 — Confirm ProductRepository extends JpaRepository

JpaRepository already inherits findAll(Pageable) from PagingAndSortingRepository (see Practical 2) — no repository changes are needed for this practical.

Step 3, 4 & 5 — Add the paginated endpoint

@RequestParam(defaultValue = ...) means a client can omit page/size/sortBy entirely and still get sensible results (page 0, 5 rows, sorted by id).

src/main/java/com/exam/p2/ProductRestController.java (new method)

```
package com.exam.p2; // extends the ProductRestController from Practical 2


import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

// ... existing @RestController @RequestMapping("/api/products") class ...

    // ---- PAGINATED + SORTED : GET /api/products/page?page=&size=&sortBy= ----
    @GetMapping("/page")
    public Page<Product> getProductsPaged(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "5") int size,
            @RequestParam(defaultValue = "productId") String sortBy) {

        // Pageable bundles "which page" + "how many rows" + "sort order"
        // into one object that Spring Data JPA understands natively.
        Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));

        // findAll(Pageable) - inherited from PagingAndSortingRepository via
        // JpaRepository (Practical 2) - runs ONE efficient SQL query with
        // LIMIT/OFFSET and ORDER BY built in; Hibernate does not fetch every
        // row into memory first.
        return productRepository.findAll(pageable);

        // The returned Page<Product> object automatically serializes to JSON
        // containing: content (the list of rows for this page), totalPages,
        // totalElements, number (current page index), size, and more.
    }
```

Step 6 & 7 — Exercise create/update/delete against the paginated view

1. Call GET /api/products/page with different page and size values and confirm totalPages/totalElements match your seeded row count.

2. POST one more product, then re-call the same page — totalElements should increase by one and the new row should appear on the correct page once sorted.

3. PUT to change a product's price so it moves to a different position when sorted by price, and confirm the page ordering updates accordingly.

4. DELETE a product and confirm totalElements decreases by one.

Postman API Testing

Practical 8 — Postman API Testing (Pagination & Sorting)

▸ 1. GET — page 0, size 5, sorted by price ascending

GET http://localhost:8080/api/products/page?page=0&size=5&sortBy=price Send

Response Status: 200 OK


```
{
  "content": [
    {
      "productId": 3,
      "productName": "Mouse",
      "price": 499.0
    },
    {
      "productId": 1,
      "productName": "Keyboard",
      "price": 899.0
    }
  ],
  "totalPages": 4,
  "totalElements": 18,
  "number": 0,
  "size": 5
}
```


▸ 2. GET — page 1 of the same query

GET http://localhost:8080/api/products/page?page=1&size=5&sortBy=price Send

Response Status: 200 OK


```
{
  "content": [
    {
      "productId": 7,
      "productName": "Monitor",
      "price": 8999.0
    }
  ],
  "totalPages": 4,
  "totalElements": 18,
  "number": 1,
  "size": 5
}
```


▸ 3. DELETE — remove a product, then re-check totalElements

DELETE http://localhost:8080/api/products/3 Send

Response Status: 200 OK


```
{
  "message": "Product deleted successfully"
}
```

Chapter 6

Building RESTful Microservices

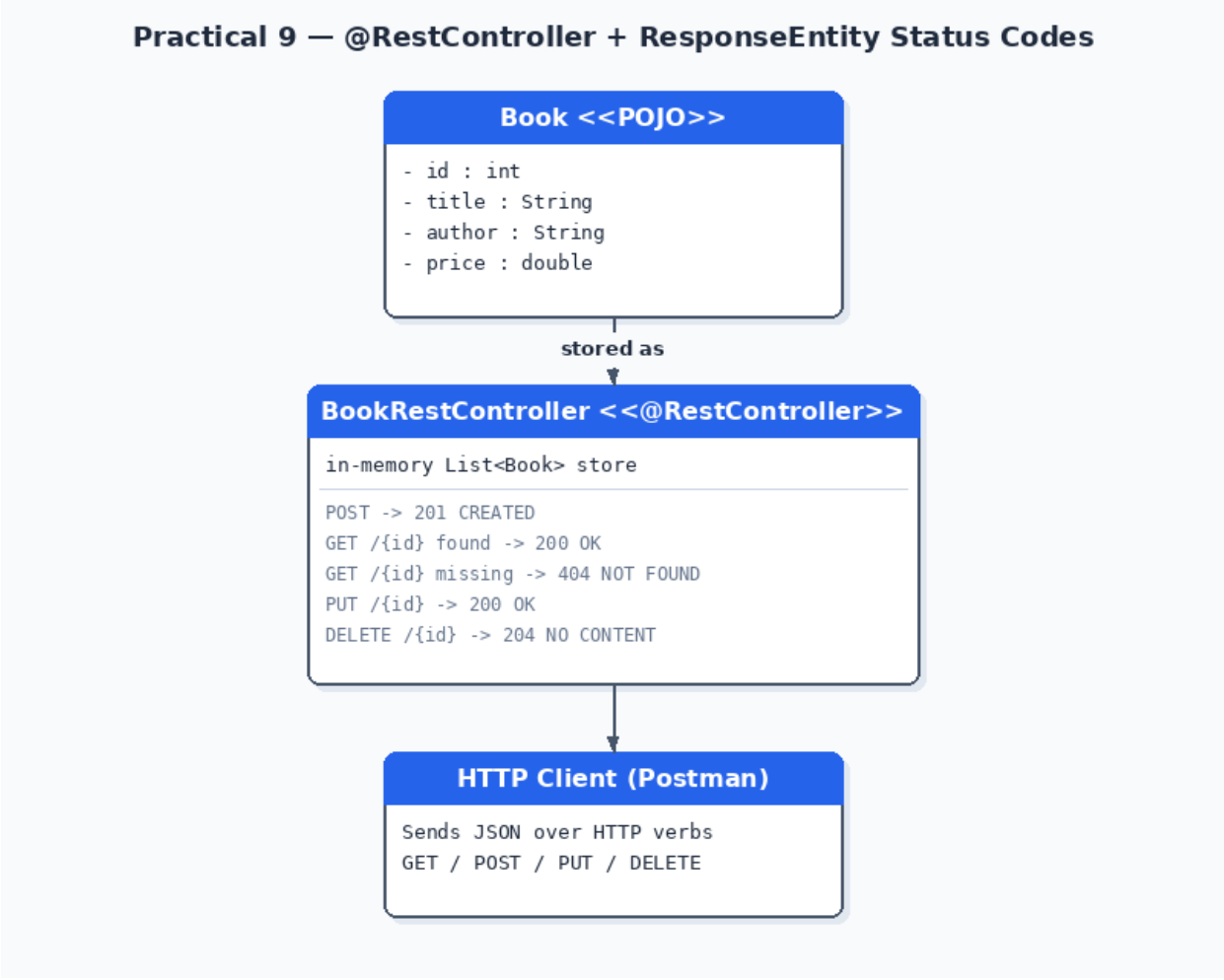
Practical No. 9 — Building a RESTful Microservice using @RestController and ResponseEntity

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To design and build a RESTful microservice endpoint using @RestController, mapping HTTP verbs to CRUD operations, and using ResponseEntity to control the HTTP status code and headers returned to the client — the core REST-building technique introduced in this chapter.

Design — Status Code Flow



> ℹ *Returning a plain object from a @RestController method always defaults to HTTP 200. Wrapping the return type in ResponseEntity<T> lets you choose the exact status code — 201 for a successful creation, 404 when a lookup fails, 204 when a delete succeeds with nothing to return, and so on.*

Step-by-Step Implementation

Step 1 — Book POJO

This practical deliberately uses a plain POJO with an in-memory list (not @Entity/JPA) so every line of the controller can focus on ResponseEntity.

src/main/java/com/exam/p9/Book.java

```
package com.exam.p9;

public class Book {
    private int id;
    private String title;
    private String author;
    private double price;

    public Book() {}
    public Book(int id, String title, String author, double price) {
        this.id = id; this.title = title; this.author = author; this.price = price;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

Step 2 to 5 — BookRestController returning ResponseEntity

src/main/java/com/exam/p9/BookRestController.java

```
package com.exam.p9;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;

/**
 * This practical intentionally uses an in-memory List instead of JPA, so the
 * focus stays entirely on ResponseEntity and HTTP status codes rather than
 * persistence (that was already covered in Practicals 1-8).
 */
@RestController
@RequestMapping("/api/books")
public class BookRestController {

    private final List<Book> books = new java.util.concurrent.CopyOnWriteArrayList<>();
    private final AtomicInteger idCounter = new AtomicInteger(1);

    // ---- CREATE : POST /api/books -> 201 CREATED ----
    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        book.setId(idCounter.getAndIncrement());
        books.add(book);
        // ResponseEntity.status(...) lets us explicitly control the status
        // code returned - 201 Created is the correct code for a successful
        // POST that creates a new resource (plain @RestController would
        // default to 200 unless we do this).
        return ResponseEntity.status(HttpStatus.CREATED).body(book);
    }

    // ---- READ ALL : GET /api/books -> 200 OK ----
    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(books);
    }
```

```
    // ---- READ ONE : GET /api/books/{id} -> 200 OK or 404 NOT FOUND ----
    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable int id) {
        return books.stream()
                .filter(b -> b.getId() == id)
                .findFirst()
                .map(ResponseEntity::ok)                // found  -> 200 OK
                .orElse(ResponseEntity.notFound().build()); // absent -> 404 NOT FOUND
    }

    // ---- UPDATE : PUT /api/books/{id} -> 200 OK or 404 NOT FOUND ----
    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable int id, @RequestBody Book updated) {
        for (Book b : books) {
            if (b.getId() == id) {
                b.setTitle(updated.getTitle());
                b.setAuthor(updated.getAuthor());
                b.setPrice(updated.getPrice());
                return ResponseEntity.ok(b);
            }
        }
        return ResponseEntity.notFound().build();
    }

    // ---- DELETE : DELETE /api/books/{id} -> 204 NO CONTENT or 404 NOT FOUND ----
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable int id) {
        boolean removed = books.removeIf(b -> b.getId() == id);
        // 204 No Content: the request succeeded but there is nothing to
        // return in the body - the standard status code for DELETE.
        return removed ? ResponseEntity.noContent().build() : ResponseEntity.notFound().build();
    }
}
```

Step 6 — Test every status code in Postman

- POST a new book and confirm Postman's status bar shows 201 Created.
- GET an id that exists → 200 OK; GET an id that does not exist → 404 Not Found.
- PUT a valid id → 200 OK; PUT a missing id → 404 Not Found.
- DELETE a valid id → 204 No Content (empty response body); DELETE it again → 404 Not Found.

Postman API Testing

Practical 9 — Postman API Testing (ResponseEntity Status Codes)

▸ 1. POST — create a book (expect 201 Created)

POST http://localhost:8080/api/books Send

Body > raw (JSON)

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 799.0
}
```

Response Status: 201 Created

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 799.0
}
```

▸ 2. GET — existing book (expect 200 OK)

GET http://localhost:8080/api/books/1 Send

Response Status: 200 OK

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 799.0
}
```

▸ 3. GET — non-existent book (expect 404 Not Found)

GET http://localhost:8080/api/books/99 Send

Response Status: 404 Not Found

```
null
```

▸ 4. DELETE — remove a book (expect 204 No Content)

DELETE http://localhost:8080/api/books/1 Send

Response Status: 204 No Content

```
null
```

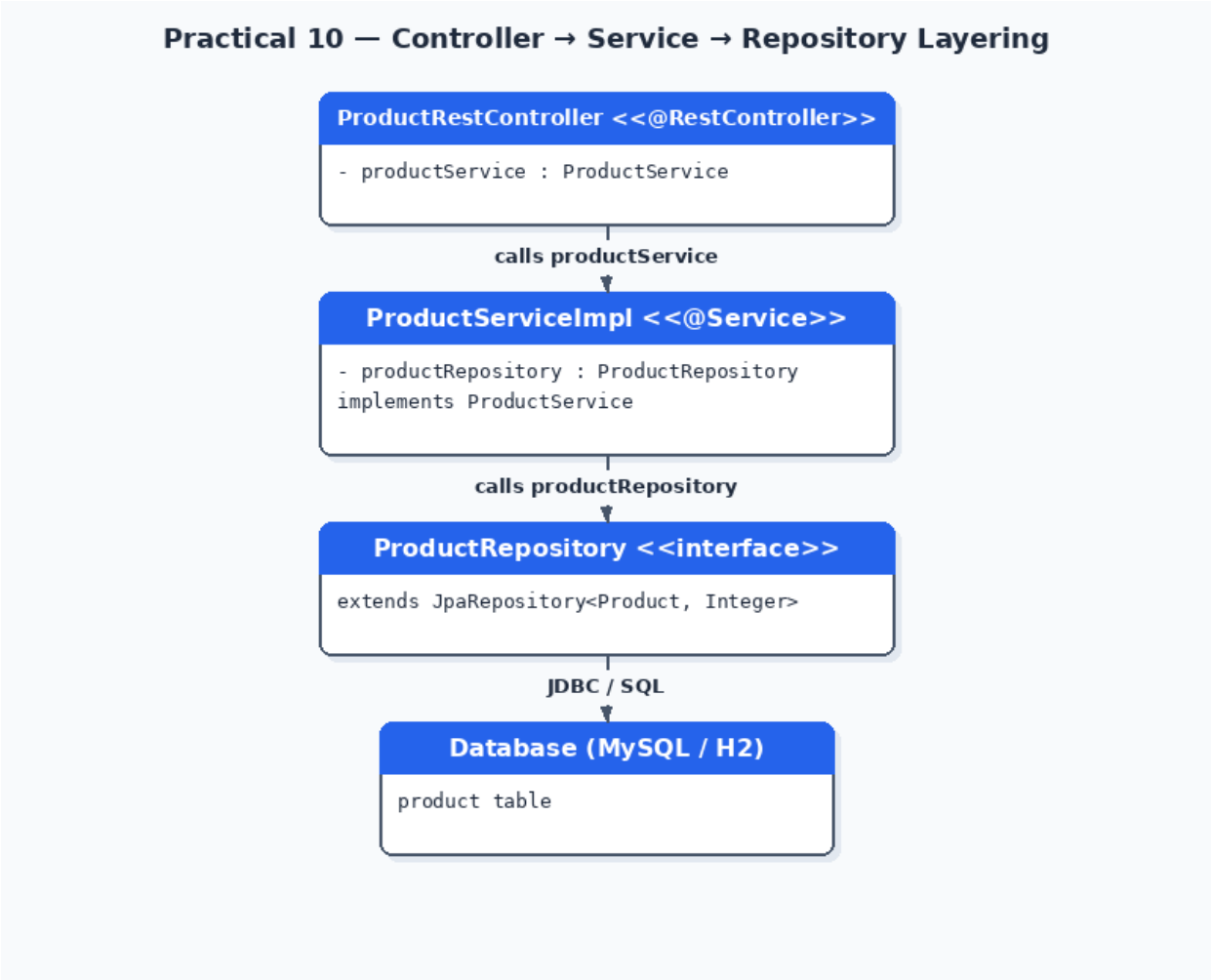
Practical No. 10 — Layered Microservice Architecture: RestController + Service + JPA Repository

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To structure a RESTful microservice into the standard Controller-Service-Repository layers recommended in this chapter, keeping business logic out of the controller and persistence access out of the service.

Design — Layered Architecture



> *i Compare this to Practical 1, where EmployeeRestController injected EmployeeRepository directly. That two-layer shortcut is fine for a small demo, but as an application grows, mixing HTTP handling and persistence access in one class makes it hard to test or reuse the business logic — hence the Service layer introduced here.*

Step-by-Step Implementation

Product and ProductRepository are identical to Practical 2 (package renamed to com.exam.p10 for isolation).

Step 1 — ProductService interface

`src/main/java/com/exam/p10/ProductService.java`

```
package com.exam.p10;

import java.util.List;

public interface ProductService {
    Product create(Product product);
    List<Product> getAll();
    Product getById(int id);
    Product update(int id, Product updated);
    void delete(int id);
}
```

Step 2 & 4 — ProductServiceImpl (business logic lives here)

src/main/java/com/exam/p10/ProductServiceImpl.java

```
package com.exam.p10;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

/**
 * @Service marks this as the business-logic layer. It is the ONLY class
 * that talks to ProductRepository - the controller below never does.
 */
@Service
public class ProductServiceImpl implements ProductService {

    @Autowired
    private ProductRepository productRepository;

    @Override
    public Product create(Product product) {
        return productRepository.save(product);
    }

    @Override
    public List<Product> getAll() {
        return productRepository.findAll();
    }

    @Override
    public Product getById(int id) {
        return productRepository.findById(id).orElse(null);
    }

    @Override
    public Product update(int id, Product updated) {
        Optional<Product> existingOpt = productRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        // Business logic (fetch -> modify -> save) lives HERE, in the
        // service, not scattered across the controller.
        Product existing = existingOpt.get();
        existing.setProductName(updated.getProductName());
        existing.setCategory(updated.getCategory());
        existing.setPrice(updated.getPrice());
        existing.setQuantity(updated.getQuantity());
        return productRepository.save(existing);
    }

    @Override
    public void delete(int id) {
        productRepository.deleteById(id);
    }
```

```
}
```

Step 3 — ProductRestController (talks only to the Service)

src/main/java/com/exam/p10/ProductRestController.java

```
package com.exam.p10;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * Notice this controller injects ONLY ProductService - never
 * ProductRepository directly. The controller's one job is to translate
 * HTTP <-> Java method calls; all logic and persistence access are
 * delegated further down the layers.
 */
@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    private ProductService productService;

    @PostMapping
    public Product create(@RequestBody Product product) {
        return productService.create(product);
    }

    @GetMapping
    public List<Product> getAll() {
        return productService.getAll();
    }

    @GetMapping("/{id}")
    public Product getById(@PathVariable int id) {
        return productService.getById(id);
    }

    @PutMapping("/{id}")
    public Product update(@PathVariable int id, @RequestBody Product updated) {
        return productService.update(id, updated);
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        productService.delete(id);
        return "Product deleted successfully";
    }
}
```

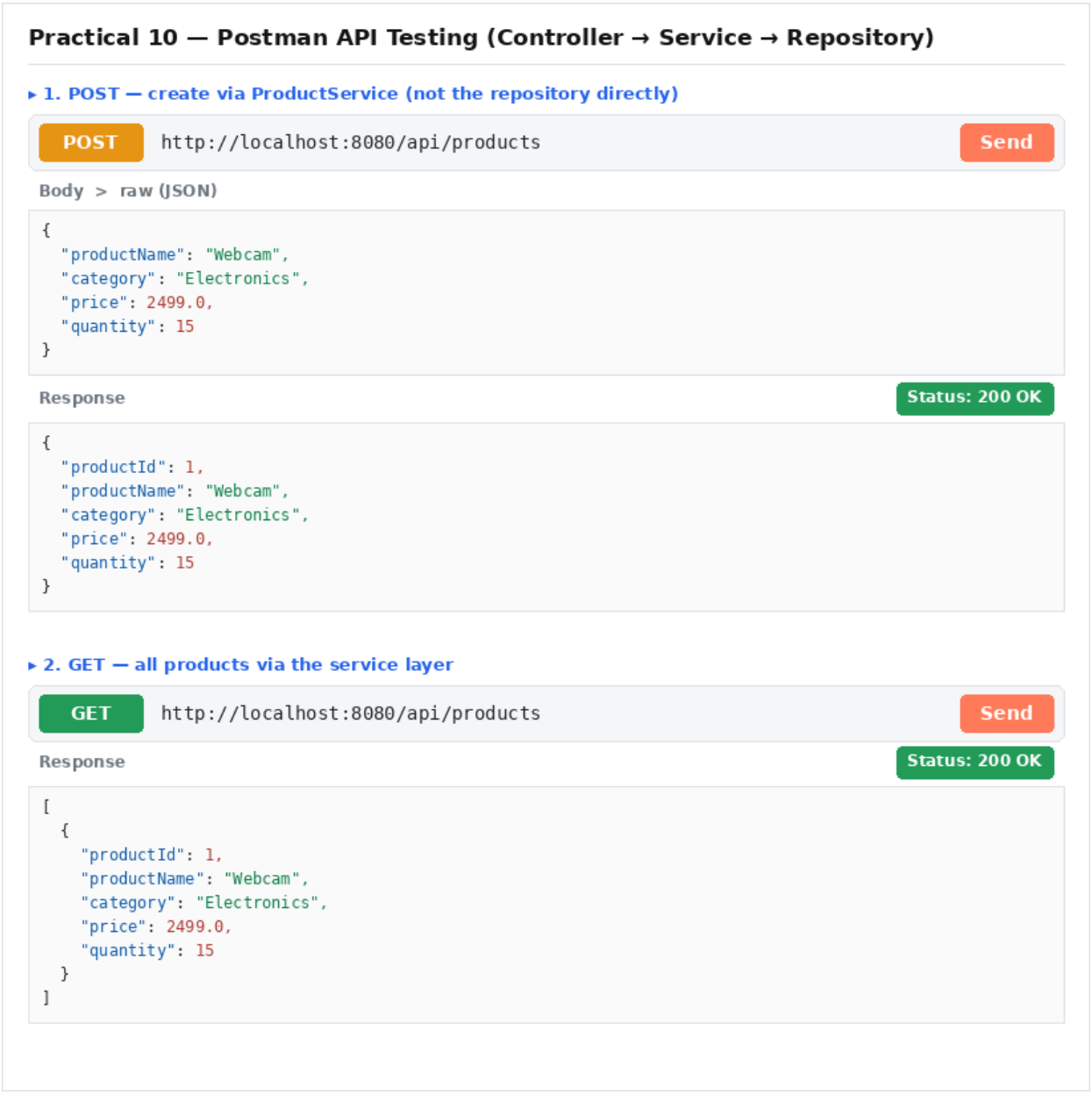
Step 5 — Confirm proper layering by code review

- ProductRestController has exactly one @Autowired field: ProductService.
- ProductRepository is @Autowired ONLY inside ProductServiceImpl.
- Update logic (fetch → modify → save) lives in the service, not the controller.

Step 6 — Test the complete CRUD lifecycle in Postman

Test POST, GET (all + by id), PUT, and DELETE exactly as in earlier practicals — from the outside, the API behaves identically; what changed is the internal structure. In Postman, save these five requests into a Collection so they can be reused/re-run together for future practicals.

Postman API Testing



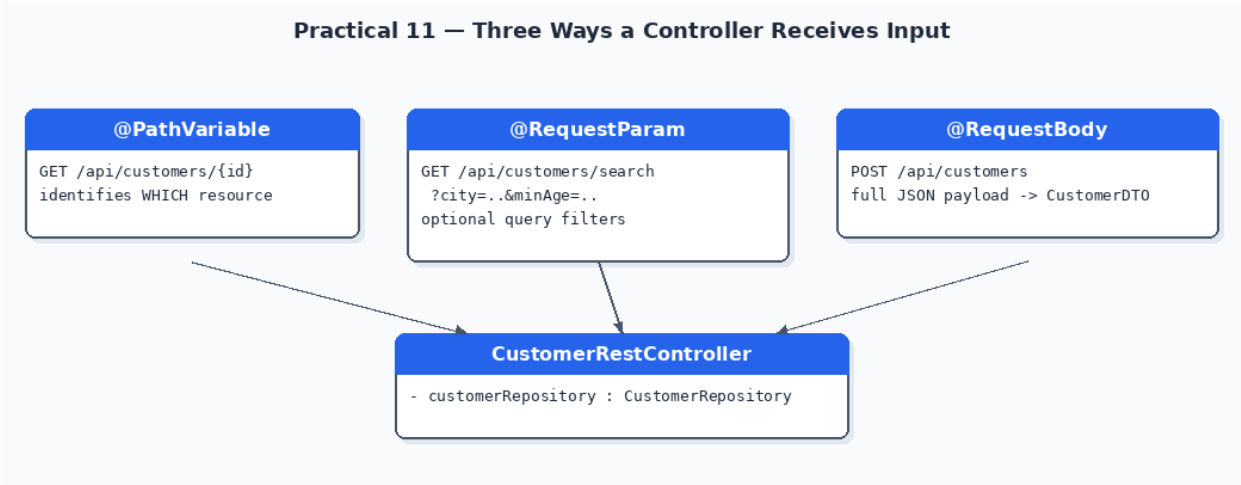
Practical No. 11 — Handling Request Data: @PathVariable, @RequestParam and @RequestBody

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To correctly use the three main ways a Spring REST controller receives client input — @PathVariable for identifying a resource, @RequestParam for optional query filters, and @RequestBody for JSON payloads.

Design — Three Input Styles



Step-by-Step Implementation

Step 1 — Customer entity and repository

src/main/java/com/exam/p11/Customer.java

```
package com.exam.p11;

import jakarta.persistence.*;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int customerId;

    private String name;
    private String city;
    private int age;

    public Customer() {}
    public Customer(String name, String city, int age) {
        this.name = name; this.city = city; this.age = age;
    }

    public int getCustomerId() { return customerId; }
    public void setCustomerId(int customerId) { this.customerId = customerId; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }
```

```
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

src/main/java/com/exam/p11/CustomerRepository.java

```
package com.exam.p11;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface CustomerRepository extends JpaRepository<Customer, Integer> {
    List<Customer> findByCityAndAgeGreaterThanEqual(String city, int age);
    List<Customer> findByCity(String city);
}
```

Step 2 to 5 — CustomerRestController exercising all three styles

- @PathVariable — the value is embedded IN the URL path itself: /api/customers/7.
- @RequestParam — values come from the query string after ?: /api/customers/search?city=Pune&minAge=25. required=false with defaultValue makes a parameter optional.
- @RequestBody — the ENTIRE JSON body of the request is deserialized into a Java object in one step.

src/main/java/com/exam/p11/CustomerRestController.java

```
package com.exam.p11;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;
import java.util.logging.Logger;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    private static final Logger log = Logger.getLogger(CustomerRestController.class.getName());

    @Autowired
    private CustomerRepository customerRepository;

    // ---- @PathVariable : identifies WHICH resource (part of the URL path) ----
    @GetMapping("/{id}")
    public Customer getById(@PathVariable int id) {
        log.info("Parameter source: @PathVariable -> id=" + id);
        return customerRepository.findById(id).orElse(null);
    }

    // ---- @RequestParam : optional query-string filters ----
    @GetMapping("/search")
    public List<Customer> search(@RequestParam String city,
                                 @RequestParam(required = false, defaultValue = "0") int minAge) {
        log.info("Parameter source: @RequestParam -> city=" + city + ", minAge=" + minAge);
        if (minAge > 0) {
            return customerRepository.findByCityAndAgeGreaterThanEqual(city, minAge);
        }
        return customerRepository.findByCity(city);
    }

    // ---- @RequestBody : full JSON payload for creating a resource ----
```

```
    @PostMapping
    public Customer create(@RequestBody Customer customer) {
        log.info("Parameter source: @RequestBody -> " + customer.getName());
        return customerRepository.save(customer);
    }

    // ---- Combines @PathVariable (which row) + @RequestBody (new values) ----
    @PutMapping("/{id}")
    public Customer update(@PathVariable int id, @RequestBody Customer updated) {
        log.info("Parameter source: @PathVariable + @RequestBody -> id=" + id);
        Optional<Customer> existingOpt = customerRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Customer existing = existingOpt.get();
        existing.setName(updated.getName());
        existing.setCity(updated.getCity());
        existing.setAge(updated.getAge());
        return customerRepository.save(existing);
    }

    // ---- @PathVariable alone is enough to identify what to delete ----
    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        log.info("Parameter source: @PathVariable -> deleting id=" + id);
        customerRepository.deleteById(id);
        return "Customer deleted successfully";
    }
}
```

Step 6 & 7 — Confirm and test each parameter source

1. In Postman, call GET /api/customers/1 — type the id directly into the URL bar (PathVariable).

2. Call GET /api/customers/search — this time use Postman's Params tab to add city and minAge as key/value rows (RequestParam).

3. Call POST /api/customers — use the Body tab (raw/JSON) to send a full customer object (RequestBody).

4. Watch the application console log after each call to confirm which parameter-source line was printed.

Postman API Testing

Practical 11 — Postman API Testing (PathVariable / RequestParam / RequestBody)

▸ 1. @PathVariable — GET by id

GET http://localhost:8080/api/customers/1 Send

Response Status: 200 OK

```
{
  "customerId": 1,
  "name": "Meera Iyer",
  "city": "Pune",
  "age": 29
}
```

▸ 2. @RequestParam — search with query params

GET http://localhost:8080/api/customers/search?city=Pune&minAge=25 Send

Response Status: 200 OK

```
[
  {
    "customerId": 1,
    "name": "Meera Iyer",
    "city": "Pune",
    "age": 29
  }
]
```

▸ 3. @RequestBody — create via full JSON payload

POST http://localhost:8080/api/customers Send

Body > raw (JSON)

```
{
  "name": "Kabir Shah",
  "city": "Mumbai",
  "age": 34
}
```

Response Status: 200 OK

```
{
  "customerId": 2,
  "name": "Kabir Shah",
  "city": "Mumbai",
  "age": 34
}
```

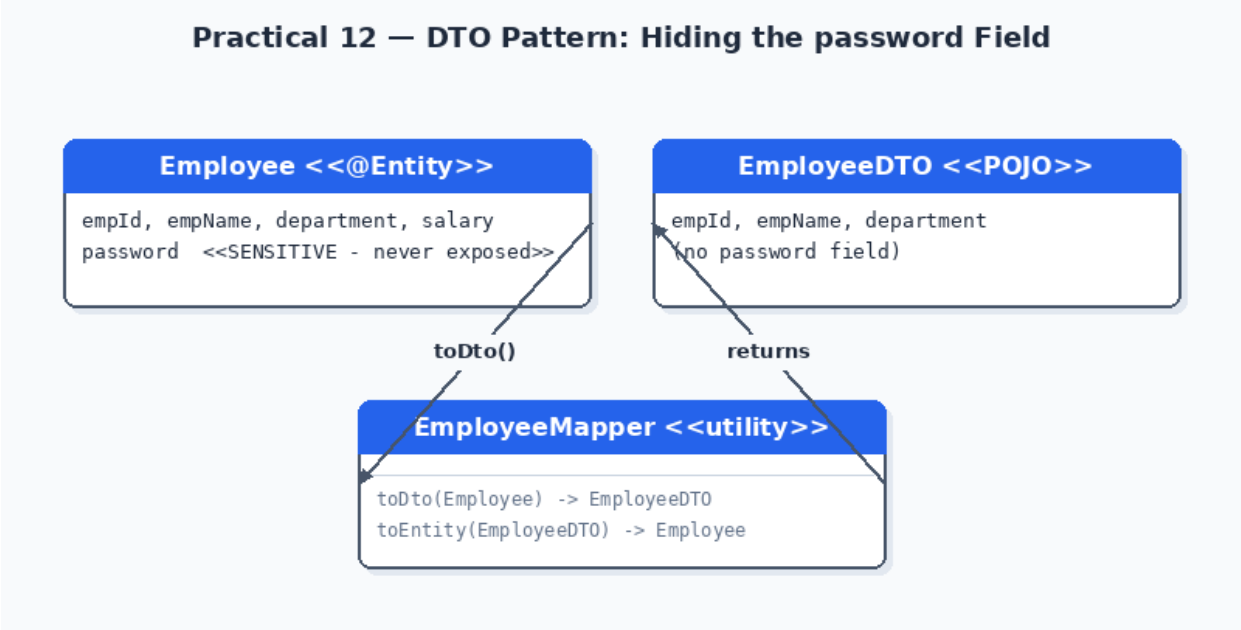
Practical No. 12 — DTO Pattern: Separating the Entity from the API Contract

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To decouple the persistent JPA entity from the data actually exposed by the REST API using the Data Transfer Object (DTO) pattern, hiding sensitive/internal fields and mapping between Entity and DTO.

Design — Entity vs DTO



> ***i*** *A common beginner mistake is to return the @Entity object directly from a controller. That couples your API's JSON shape to your database schema, and — as this practical shows — can leak sensitive fields like password that were only ever meant for internal use.*

Step-by-Step Implementation

Step 1 — Add a sensitive field to Employee

src/main/java/com/exam/p12/Employee.java

```
package com.exam.p12;

import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int empId;

    private String empName;
    private String department;
    private double salary;

    // Sensitive field: this must NEVER be returned by any REST endpoint.
    private String password;
```

```
    public Employee() {}

    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

Step 2 — EmployeeDTO (the public contract)

src/main/java/com/exam/p12/EmployeeDTO.java

```
package com.exam.p12;

/**
 * EmployeeDTO (Data Transfer Object) - the shape of data the OUTSIDE WORLD
 * is allowed to see. Notice password and salary are intentionally absent;
 * only fields safe for public API consumption are included.
 */
public class EmployeeDTO {
    private int empId;
    private String empName;
    private String department;

    public EmployeeDTO() {}
    public EmployeeDTO(int empId, String empName, String department) {
        this.empId = empId; this.empName = empName; this.department = department;
    }

    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}
```

Step 3 — EmployeeMapper

src/main/java/com/exam/p12/EmployeeMapper.java

```
package com.exam.p12;

public class EmployeeMapper {

    // Entity -> DTO : strips out password (and any other internal-only fields)
    public static EmployeeDTO toDto(Employee e) {
        return new EmployeeDTO(e.getEmpId(), e.getEmpName(), e.getDepartment());
    }

    // DTO -> Entity : used when creating; password is set separately if needed
    public static Employee toEntity(EmployeeDTO d) {
        Employee e = new Employee();
        e.setEmpId(d.getEmpId());
        e.setEmpName(d.getEmpName());
        e.setDepartment(d.getDepartment());
        return e;
    }
}
```

Step 4, 5 & 6 — Controller converts at every boundary

EmployeeRepository here is the same CrudRepository<Employee, Integer> pattern from Practical 1 - only the controller changes.

src/main/java/com/exam/p12/EmployeeRestController.java

```
package com.exam.p12;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    // ---- CREATE : accepts a DTO, converts to entity, saves ----
    @PostMapping
    public EmployeeDTO create(@RequestBody EmployeeDTO dto) {
        Employee entity = EmployeeMapper.toEntity(dto);
        Employee saved = employeeRepository.save(entity);
        return EmployeeMapper.toDto(saved); // convert back -> password never leaves the server
    }

    // ---- READ ALL : entities are mapped to DTOs before being returned ----
    @GetMapping
    public List<EmployeeDTO> getAll() {
        return employeeRepository.findAll().stream()
                .map(EmployeeMapper::toDto)
                .collect(Collectors.toList());
    }

    // ---- READ ONE ----
    @GetMapping("/{id}")
    public EmployeeDTO getById(@PathVariable int id) {
        return employeeRepository.findById(id).map(EmployeeMapper::toDto).orElse(null);
    }

    // ---- UPDATE : maps the incoming DTO onto the EXISTING entity, so the
    //      real (unseen) password field is left untouched ----
    @PutMapping("/{id}")
    public EmployeeDTO update(@PathVariable int id, @RequestBody EmployeeDTO dto) {
        Optional<Employee> existingOpt = employeeRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Employee existing = existingOpt.get();
        existing.setEmpName(dto.getEmpName());
        existing.setDepartment(dto.getDepartment());
        // existing.password is NEVER overwritten here - the DTO never carried it.
        Employee saved = employeeRepository.save(existing);
        return EmployeeMapper.toDto(saved);
    }

    // ---- DELETE ----
    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        employeeRepository.deleteById(id);
        return "Employee deleted successfully";
    }
}
```

Step 7 — Verify the password never appears

1. POST a new employee via the DTO shape (empName, department only).

2. Manually set a password value directly in the database (H2 console or MySQL Workbench) to simulate a real password being on record.

3. Call GET /api/employees and GET /api/employees/{id} and confirm the JSON response never contains a password key.

Postman API Testing

Practical 12 — Postman API Testing (DTO Hides password)

▸ 1. POST — create employee via EmployeeDTO

POST http://localhost:8080/api/employees Send

Body > raw (JSON)

```
{
  "empName": "Farhan Ali",
  "department": "IT"
}
```

Response Status: 200 OK

```
{
  "empId": 1,
  "empName": "Farhan Ali",
  "department": "IT"
}
```

▸ 2. GET — confirm the password field never appears in the response

GET http://localhost:8080/api/employees Send

Response Status: 200 OK

```
[
  {
    "empId": 1,
    "empName": "Farhan Ali",
    "department": "IT"
  }
]
```

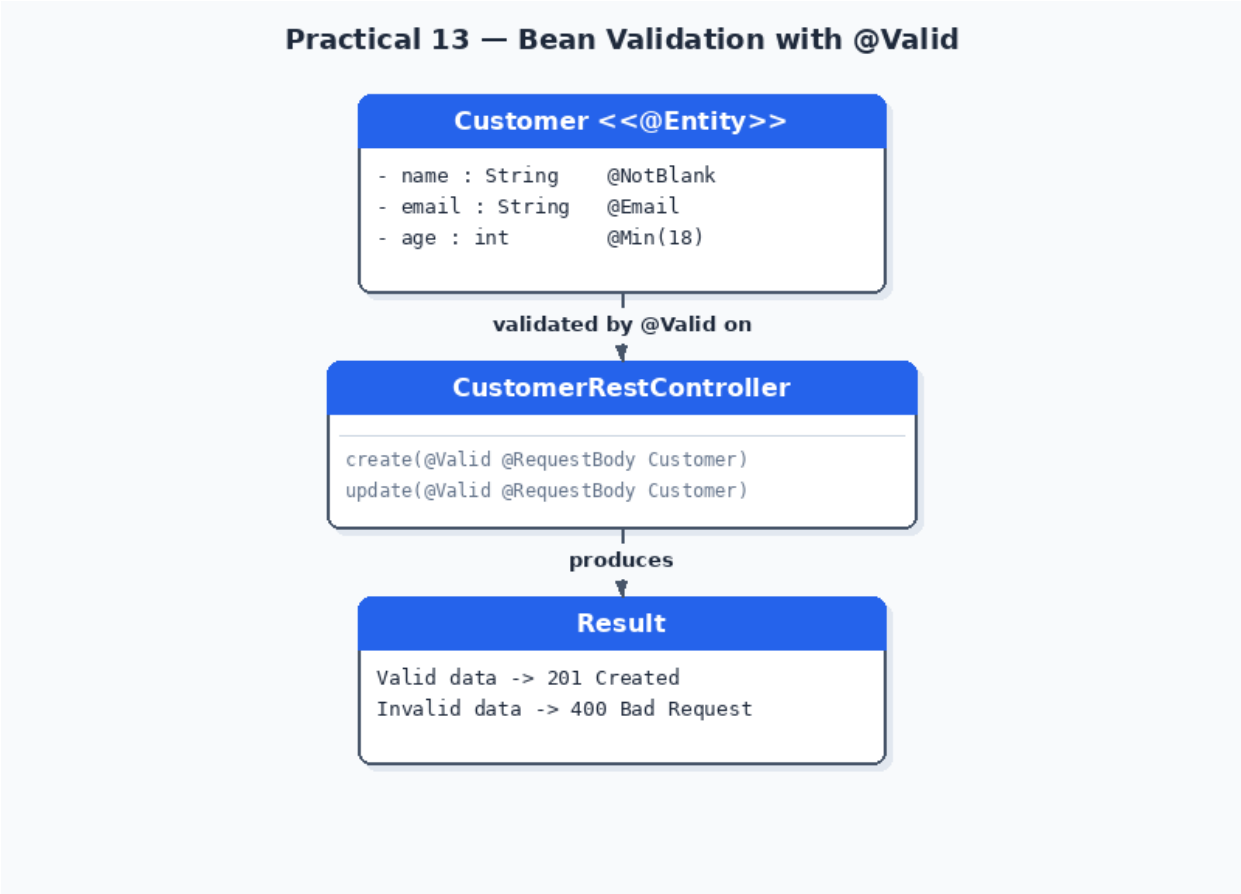
Practical No. 13 — Request Validation using Bean Validation (@Valid) with RestController

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To validate incoming JSON payloads declaratively using Jakarta Bean Validation annotations (@NotBlank, @Email, @Min) combined with @Valid on the controller method.

Design — Validation Flow



Step-by-Step Implementation

Step 1 — Add spring-boot-starter-validation

Already included in the common pom.xml from the one-time setup section.

Step 2 — Annotate Customer's fields

src/main/java/com/exam/p13/Customer.java

```
package com.exam.p13;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
```

```
@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int customerId;

    @NotBlank(message = "name must not be blank")
    private String name;

    @Email(message = "email must be a well-formed email address")
    private String email;

    @Min(value = 18, message = "age must be greater than or equal to 18")
    private int age;

    public Customer() {}

    public int getCustomerId() { return customerId; }
    public void setCustomerId(int customerId) { this.customerId = customerId; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}
```

src/main/java/com/exam/p13/CustomerRepository.java

```
package com.exam.p13;

import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends JpaRepository<Customer, Integer> {
}
```

Step 3 to 6 — Add @Valid to the controller's create and update methods

src/main/java/com/exam/p13/CustomerRestController.java

```
package com.exam.p13;

import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Optional;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    @Autowired
    private CustomerRepository customerRepository;

    // @Valid tells Spring: run Bean Validation on the deserialized Customer
    // BEFORE this method body executes. If validation fails, Spring throws
    // MethodArgumentNotValidException automatically and this method body
    // never runs at all - by default that becomes an HTTP 400 response.
    @PostMapping
    public ResponseEntity<Customer> create(@Valid @RequestBody Customer customer) {
        Customer saved = customerRepository.save(customer);
```

```
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @GetMapping("/{id}")
    public Customer getById(@PathVariable int id) {
        return customerRepository.findById(id).orElse(null);
    }

    // @Valid on update too, so a bad edit is rejected exactly like a bad create.
    @PutMapping("/{id}")
    public Customer update(@PathVariable int id, @Valid @RequestBody Customer updated) {
        Optional<Customer> existingOpt = customerRepository.findById(id);
        if (existingOpt.isEmpty()) return null;
        Customer existing = existingOpt.get();
        existing.setName(updated.getName());
        existing.setEmail(updated.getEmail());
        existing.setAge(updated.getAge());
        return customerRepository.save(existing);
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id) {
        customerRepository.deleteById(id);
        return "Customer deleted successfully";
    }
}
```

> ***i*** *At this stage, Spring's DEFAULT validation-failure response is a fairly raw 400 Bad Request with a generic error body. Practical 14 (@ControllerAdvice) replaces this with a clean, structured ErrorResponse listing exactly which fields failed and why.*

Step 7 — Test invalid then valid payloads

1. POST a Customer with a blank name, malformed email, and age below 18 — confirm a 400 Bad Request.
2. POST a fully valid Customer — confirm 201 Created.
3. PUT an invalid update against the newly created id and confirm it is also rejected with 400.
4. DELETE the customer and confirm success.

Postman API Testing

Practical 13 — Postman API Testing (@Valid Bean Validation)

▸ 1. POST — invalid payload (blank name, bad email, age < 18)

POST http://localhost:8080/api/customers Send

Body > raw (JSON)

```
{
  "name": "",
  "email": "not-an-email",
  "age": 15
}
```

Response Status: 400 Bad Request

```
{
  "status": 400,
  "errors": [
    "name: must not be blank",
    "email: must be a well-formed email address",
    "age: must be greater than or equal to 18"
  ]
}
```

▸ 2. POST — valid payload

POST http://localhost:8080/api/customers Send

Body > raw (JSON)

```
{
  "name": "Divya Menon",
  "email": "divya@example.com",
  "age": 24
}
```

Response Status: 201 Created

```
{
  "customerId": 1,
  "name": "Divya Menon",
  "email": "divya@example.com",
  "age": 24
}
```

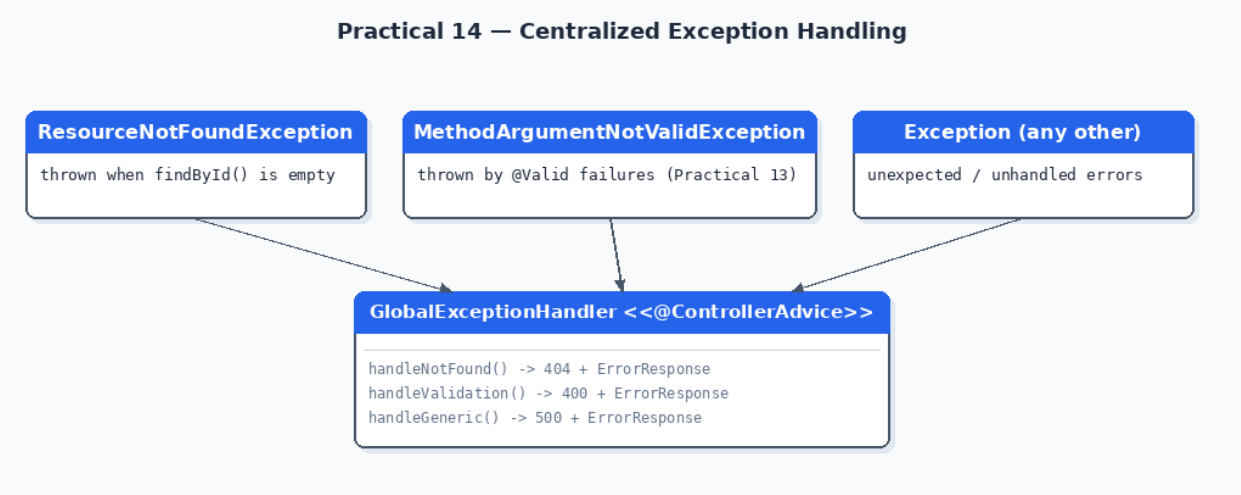
Practical No. 14 — Global Exception Handling using @ControllerAdvice and @ExceptionHandler

Textbook Reference: Chapter 6 — Building RESTful Microservices

Objective

To centralize exception handling across all REST controllers using @ControllerAdvice, returning a consistent, structured error response (timestamp, message, status, path) instead of default Spring stack-trace pages.

Design — Exception Routing



Step-by-Step Implementation

Step 1 & 2 — Custom exception and ErrorResponse POJO

src/main/java/com/exam/p14/ResourceNotFoundException.java

```
package com.exam.p14;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

src/main/java/com/exam/p14/ErrorResponse.java

```
package com.exam.p14;

import java.time.LocalDateTime;

public class ErrorResponse {
    private LocalDateTime timestamp;
    private String message;
    private int status;
    private String path;

    public ErrorResponse(LocalDateTime timestamp, String message, int status, String path) {
        this.timestamp = timestamp;
        this.message = message;
        this.status = status;
        this.path = path;
    }
```

```
    public LocalDateTime getTimestamp() { return timestamp; }
    public String getMessage() { return message; }
    public int getStatus() { return status; }
    public String getPath() { return path; }
}
```

Step 3 — Throw ResourceNotFoundException instead of returning null

Go back to any earlier practical's "fetch by id" logic (e.g. Practical 1/12's Employee service) and replace .orElse(null) with .orElseThrow(...):

Employee service - updated getById()

```
// Inside EmployeeServiceImpl (or directly in the controller) - Practicals 1/12:
public Employee getById(int id) {
    return employeeRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee not found with id " + id));
    // Previously this returned null on a miss; now GlobalExceptionHandler
    // turns this exception into a clean structured 404 automatically.
}
```

Step 4, 5 & 6 — GlobalExceptionHandler

src/main/java/com/exam/p14/GlobalExceptionHandler.java

```
package com.exam.p14;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;

import java.time.LocalDateTime;
import java.util.stream.Collectors;

/**
 * @ControllerAdvice makes every @ExceptionHandler method here apply
 * GLOBALLY, across every @RestController in the application - we no longer
 * need try/catch blocks scattered through individual controller methods.
 */
@ControllerAdvice
public class GlobalExceptionHandler {

    // ---- Handles our custom "row not found" exception -> 404 ----
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex, WebRequest request) {
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(), ex.getMessage(), HttpStatus.NOT_FOUND.value(),
                request.getDescription(false).replace("uri=", ""));
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    // ---- Handles @Valid failures from Practical 13 -> 400, with field detail ----
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex, WebRequest request)
{
        String combinedMessage = ex.getBindingResult().getFieldErrors().stream()
                .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
                .collect(Collectors.joining("; "));
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(), combinedMessage, HttpStatus.BAD_REQUEST.value(),
                request.getDescription(false).replace("uri=", ""));
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
```

```
    // ---- Catch-all for anything else unhandled -> 500 ----
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex, WebRequest request) {
        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(), "An unexpected error occurred: " + ex.getMessage(),
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                request.getDescription(false).replace("uri=", ""));
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Step 7 — Route every verb through ResourceNotFoundException

Update PUT /api/employees/{id} and DELETE /api/employees/{id} to also throw ResourceNotFoundException (instead of silently doing nothing / returning null) when the id does not exist, so all four HTTP verbs behave consistently against a missing id.

Test each handler

1. GET a non-existent employee id → structured JSON 404 (not a raw stack trace).
2. POST an invalid Customer payload (Practical 13) → structured JSON 400 listing every failed field.
3. Trigger any other runtime error (e.g. malformed JSON) → structured JSON 500 from the catch-all handler.

Postman API Testing

Practical 14 — Postman API Testing (Global @ControllerAdvice)

▸ 1. GET — non-existent employee → structured 404 (not a stack trace)

GET `http://localhost:8080/api/employees/99` **Send**

Response **Status: 404 Not Found**

```
{
  "timestamp": "2026-08-22T10:15:30",
  "message": "Employee not found with id 99",
  "status": 404,
  "path": "/api/employees/99"
}
```

▸ 2. POST — invalid payload → structured 400 with field errors

POST `http://localhost:8080/api/customers` **Send**

Body > raw (JSON)

```
{
  "name": "",
  "email": "bad",
  "age": 10
}
```

Response **Status: 400 Bad Request**

```
{
  "timestamp": "2026-08-22T10:16:02",
  "message": "Validation failed",
  "status": 400,
  "path": "/api/customers"
}
```

Chapter 7

Securing a Web Application

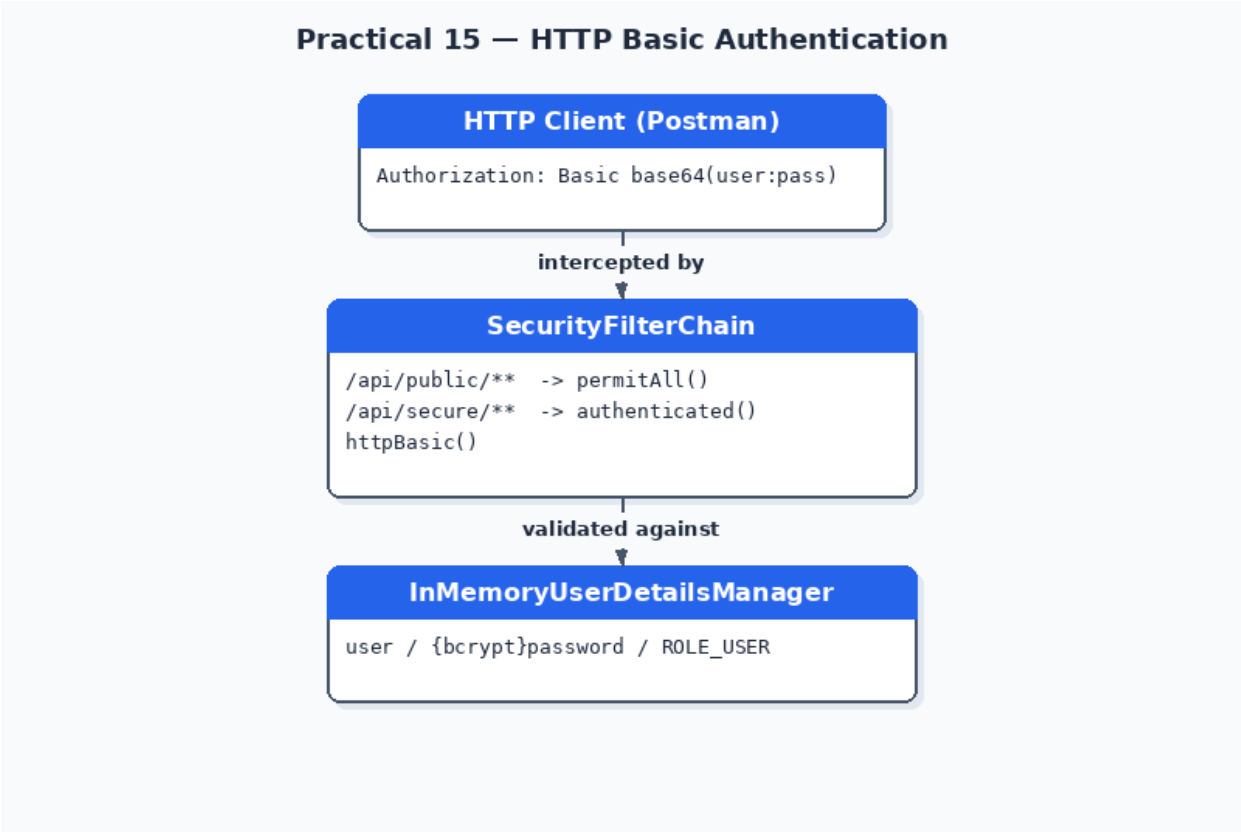
Practical No. 15 — Securing REST API with HTTP Basic Authentication (Spring Security)

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To add Spring Security to a Spring Boot REST application, configure in-memory users with encoded passwords, and secure selected endpoints so that only authenticated requests can access them.

Design — Basic Auth Flow



> ***i*** *The moment spring-boot-starter-security is on the classpath, Spring Security locks down EVERY endpoint by default. SecurityConfig below is what selectively opens up /api/public/** while keeping /api/secure/** protected.*

Step-by-Step Implementation

Step 1 — Dependency

spring-boot-starter-security is already included in the common pom.xml.

Step 2, 3 & 4 — SecurityConfig

src/main/java/com/exam/p15/SecurityConfig.java

```
package com.exam.p15;


import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
```

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
public class SecurityConfig {

    // BCrypt is a one-way, salted hashing algorithm - passwords are never
    // stored or compared as plain text, even in memory.
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // A simple in-memory user store - good for learning; Practical 16
    // upgrades this to a real database-backed user store.
    @Bean
    public InMemoryUserDetailsManager userDetailsService(PasswordEncoder encoder) {
        UserDetails user = User.withUsername("user")
                .password(encoder.encode("password"))
                .roles("USER")
                .build();
        return new InMemoryUserDetailsManager(user);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()   // no login required
                .requestMatchers("/api/secure/**").authenticated() // login required
                .anyRequest().authenticated()
            )
            .httpBasic(httpBasic -> {}) // enables the HTTP Basic Authentication scheme
            .csrf(csrf -> csrf.disable()); // simplifies REST API testing with Postman
        return http.build();
    }
}
```

Step 5 & 6 — Public and secured endpoints, full CRUD on the secured side

src/main/java/com/exam/p15/DataController.java

```
package com.exam.p15;

import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

@RestController
public class DataController {

    // ---- UNSECURED : GET /api/public/data ----
    @GetMapping("/api/public/data")
    public Map<String, String> publicData() {
        return Map.of("message", "This is public data - no login needed");
    }

    // In-memory "Note" resource, fully protected by /api/secure/** below
    private final List<Map<String, Object>> notes = new CopyOnWriteArrayList<>();
    private final AtomicInteger idCounter = new AtomicInteger(1);
```

```
    // ---- SECURED : GET /api/secure/data ----
    @GetMapping("/api/secure/data")
    public Map<String, Object> secureGet() {
        return Map.of("message", "This is protected data, hello user!");
    }

    // ---- SECURED : POST /api/secure/data ----
    @PostMapping("/api/secure/data")
    public Map<String, Object> secureCreate(@RequestBody Map<String, String> body) {
        Map<String, Object> note = Map.of("id", idCounter.getAndIncrement(), "text", body.get("text"));
        notes.add(note);
        return note;
    }

    // ---- SECURED : PUT /api/secure/data/{id} ----
    @PutMapping("/api/secure/data/{id}")
    public String secureUpdate(@PathVariable int id, @RequestBody Map<String, String> body) {
        return "Note " + id + " updated to: " + body.get("text");
    }

    // ---- SECURED : DELETE /api/secure/data/{id} ----
    @DeleteMapping("/api/secure/data/{id}")
    public String secureDelete(@PathVariable int id) {
        notes.removeIf(n -> n.get("id").equals(id));
        return "Note " + id + " deleted";
    }
}
```

Step 7 — Test with and without credentials

1. Call GET /api/public/data with no credentials → 200 OK.

2. Call GET /api/secure/data with no credentials → 401 Unauthorized.

3. In Postman's Authorization tab, choose Basic Auth and enter username user, password password, then re-call GET /api/secure/data → 200 OK.

4. Repeat the same Basic Auth setup for POST, PUT and DELETE on /api/secure/data and confirm each succeeds only with credentials attached.

Postman API Testing

Practical 15 — Postman API Testing (HTTP Basic Auth)

▸ 1. GET secure endpoint WITHOUT credentials → 401

GET http://localhost:8080/api/secure/data Send

Response Status: 401 Unauthorized

```
{
  "error": "Unauthorized",
  "status": 401
}
```

▸ 2. GET secure endpoint WITH Basic Auth (user / password) → 200

GET http://localhost:8080/api/secure/data Send

Authorization: Basic Auth (user : password)

Response Status: 200 OK

```
{
  "message": "This is protected data, hello user!"
}
```

▸ 3. POST secure resource WITH Basic Auth → 200

POST http://localhost:8080/api/secure/data Send

Authorization: Basic Auth (user : password)

Body > raw (JSON)

```
{
  "text": "Remember to submit the assignment"
}
```

Response Status: 200 OK

```
{
  "id": 1,
  "text": "Remember to submit the assignment"
}
```

> ***i*** *In Postman: open the Authorization tab of a request, set Type to Basic Auth, and enter the username/password — Postman then base64-encodes them into the Authorization header automatically.*

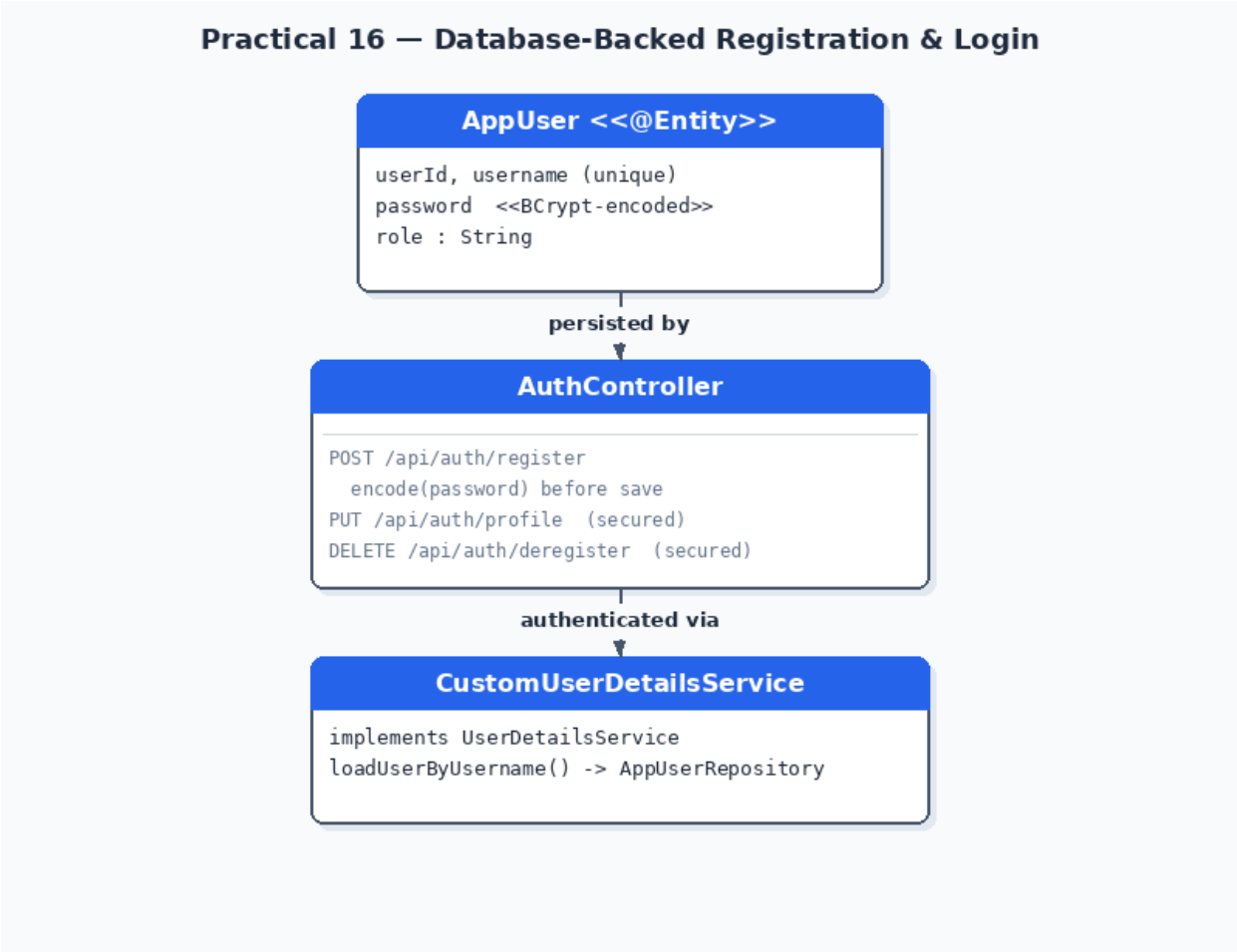
Practical No. 16 — User Registration and Login with BCrypt Password Encoding

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To implement a database-backed user registration and login mechanism, storing passwords securely using BCryptPasswordEncoder instead of plain text, and authenticating against the database via a custom UserDetailsService.

Design — Registration & Login Flow



Step-by-Step Implementation

Step 1 — AppUser entity and repository

src/main/java/com/exam/p16/AppUser.java

```
package com.exam.p16;

import jakarta.persistence.*;

@Entity
public class AppUser {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;

    @Column(unique = true)
    private String username;

    private String password; // stored as a BCrypt hash, never plain text
    private String role;

    public AppUser() {}
    public AppUser(String username, String password, String role) {
        this.username = username; this.password = password; this.role = role;
    }

    public int getUserId() { return userId; }
    public void setUserId(int userId) { this.userId = userId; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

src/main/java/com/exam/p16/AppUserRepository.java

```
package com.exam.p16;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface AppUserRepository extends JpaRepository<AppUser, Integer> {
    Optional<AppUser> findByUsername(String username);
}
```

Step 2 — CustomUserDetailsService

src/main/java/com/exam/p16/CustomUserDetailsService.java

```
package com.exam.p16;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private AppUserRepository appUserRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser appUser = appUserRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

        // Build a Spring Security User from OUR stored (already-encoded)
        // password and role - Spring Security compares the raw password the
        // client sends against this stored hash using BCrypt automatically.
        return User.withUsername(appUser.getUsername())
                .password(appUser.getPassword())
```

```
                .roles(appUser.getRole())
                .build();
    }
}
```

Step 3 — Update SecurityConfig to use the database

src/main/java/com/exam/p16/SecurityConfig.java

```
package com.exam.p16;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // Wires our CustomUserDetailsService + PasswordEncoder together, replacing
    // the InMemoryUserDetailsManager from Practical 15.
    @Bean
    public DaoAuthenticationProvider authenticationProvider(CustomUserDetailsService uds, PasswordEncoder encoder)
{
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(uds);
        provider.setPasswordEncoder(encoder);
        return provider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/register").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic(httpBasic -> {})
            .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

Step 4, 6 & 7 — AuthController: register, update profile, deregister

src/main/java/com/exam/p16/AuthController.java

```
package com.exam.p16;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.Optional;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AppUserRepository appUserRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    // ---- REGISTER : POST /api/auth/register (public) ----
    @PostMapping("/register")
    public Map<String, Object> register(@RequestBody AppUser user) {
        // Encode the incoming plain-text password BEFORE saving - this is
        // the single most important line in the whole practical.
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        AppUser saved = appUserRepository.save(user);
        return Map.of("userId", saved.getUserId(), "username", saved.getUsername(),
                      "role", saved.getRole(), "message", "User registered successfully");
    }

    // ---- UPDATE OWN PROFILE : PUT /api/auth/profile (secured) ----
    @PutMapping("/profile")
    public AppUser updateProfile(Authentication authentication, @RequestBody Map<String, String> body) {
        String username = authentication.getName(); // identity from validated Basic Auth
        Optional<AppUser> existingOpt = appUserRepository.findByUsername(username);
        if (existingOpt.isEmpty()) return null;
        AppUser existing = existingOpt.get();
        if (body.containsKey("username")) existing.setUsername(body.get("username"));
        if (body.containsKey("role")) existing.setRole(body.get("role"));
        return appUserRepository.save(existing);
    }

    // ---- DEREGISTER : DELETE /api/auth/deregister (secured) ----
    @DeleteMapping("/deregister")
    public String deregister(Authentication authentication) {
        String username = authentication.getName();
        appUserRepository.findByUsername(username).ifPresent(u -> appUserRepository.deleteById(u.getUserId()));
        return "Account deregistered successfully";
    }
}
```

Step 5 — Confirm the stored password is hashed

1. Register a new user via POST /api/auth/register with a plain-text password.

2. Open your database (H2 console or MySQL Workbench) and SELECT * FROM app_user - the password column should show a long $2a$10$... BCrypt hash, never the original text.

Step 8 — Test login and secured operations

1. Use Basic Auth in Postman with the ORIGINAL plain-text password (not the hash) against any secured endpoint — Spring Security hashes the incoming password internally and compares it to the stored hash.

2. Call PUT /api/auth/profile (secured) to change your own username, then DELETE /api/auth/deregister to remove the account.

Postman API Testing

Practical 16 — Postman API Testing (DB-Backed Registration & Login)

▸ 1. POST — register a new user (password will be BCrypt-encoded)

POST `http://localhost:8080/api/auth/register` **Send**

Body > raw (JSON)

```
{
  "username": "priya99",
  "password": "MySecret123",
  "role": "USER"
}
```

Response **Status: 200 OK**

```
{
  "userId": 1,
  "username": "priya99",
  "role": "USER",
  "message": "User registered successfully"
}
```

▸ 2. GET secure endpoint using Basic Auth with the ORIGINAL plain-text password

GET `http://localhost:8080/api/secure/data` **Send**

Authorization: Basic Auth (priya99 : MySecret123)

Response **Status: 200 OK**

```
{
  "message": "This is protected data, hello priya99!"
}
```

▸ 3. PUT — update own profile (secured)

PUT `http://localhost:8080/api/auth/profile` **Send**

Authorization: Basic Auth (priya99 : MySecret123)

Body > raw (JSON)

```
{
  "username": "priya_new",
  "role": "USER"
}
```

Response **Status: 200 OK**

```
{
  "userId": 1,
  "username": "priya_new",
  "role": "USER"
}
```

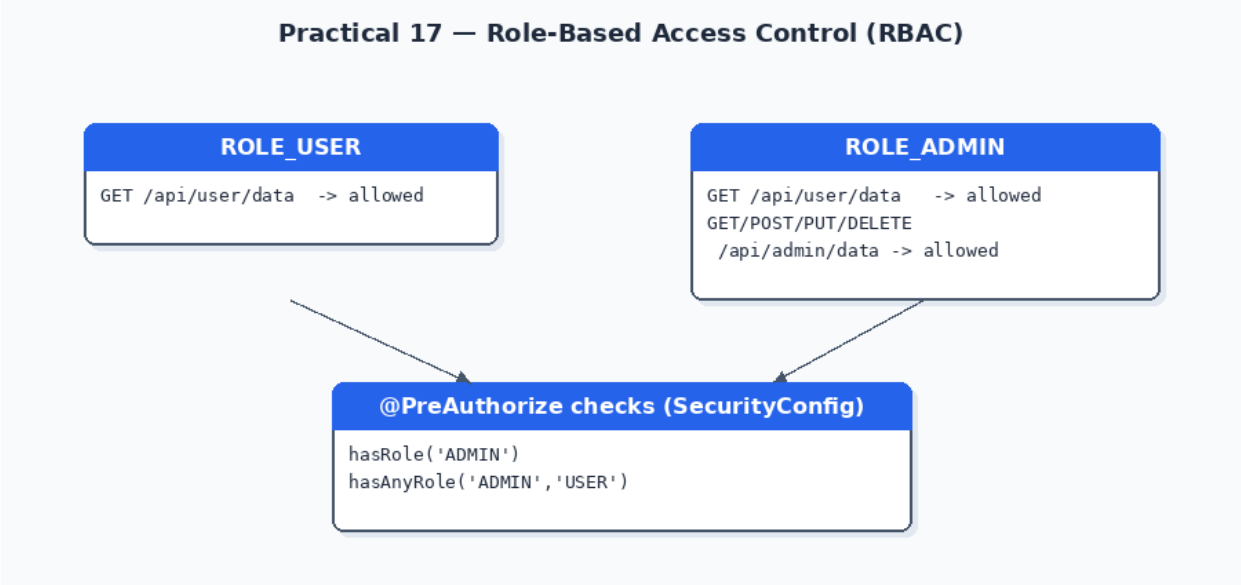
Practical No. 17 — Role-Based Access Control (RBAC) with Spring Security

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To extend Spring Security configuration for role-based authorization, defining users with different roles (ADMIN, USER) and restricting specific endpoints using @PreAuthorize.

Design — Role Matrix



Step-by-Step Implementation

Step 1 & 2 — Two users, two roles, @EnableMethodSecurity

src/main/java/com/exam/p17/SecurityConfig.java

```
package com.exam.p17;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

/**
 * @EnableMethodSecurity turns on support for @PreAuthorize annotations
 * directly on controller methods, in addition to (or instead of) the URL
 * pattern matching used in earlier practicals.
 */
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
```

```
        return new BCryptPasswordEncoder();
    }

    @Bean
    public InMemoryUserDetailsManager userDetailsService(PasswordEncoder encoder) {
        UserDetails normalUser = User.withUsername("normaluser")
                .password(encoder.encode("password")).roles("USER").build();
        UserDetails adminUser = User.withUsername("adminuser")
                .password(encoder.encode("password")).roles("ADMIN").build();
        return new InMemoryUserDetailsManager(normalUser, adminUser);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
            .httpBasic(httpBasic -> {})
            .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

Step 3, 4 & 5 — @PreAuthorize on individual methods

hasRole('ADMIN') requires exactly the ADMIN role; hasAnyRole('ADMIN','USER') allows either. Spring automatically prefixes the role name with ROLE_ internally, so .roles("ADMIN") in SecurityConfig matches hasRole('ADMIN') here.

src/main/java/com/exam/p17/RoleBasedController.java

```
package com.exam.p17;

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

@RestController
public class RoleBasedController {

    private final List<Map<String, Object>> adminItems = new CopyOnWriteArrayList<>();
    private final AtomicInteger idCounter = new AtomicInteger(1);

    // ---- ADMIN ONLY : GET /api/admin/data ----
    @GetMapping("/api/admin/data")
    @PreAuthorize("hasRole('ADMIN')")
    public Map<String, String> getAdminData() {
        return Map.of("message", "Welcome Admin, here is the sensitive data");
    }

    // ---- ADMIN ONLY : POST /api/admin/data ----
    @PostMapping("/api/admin/data")
    @PreAuthorize("hasRole('ADMIN')")
    public Map<String, Object> createAdminItem(@RequestBody Map<String, String> body) {
        Map<String, Object> item = Map.of("id", idCounter.getAndIncrement(), "text", body.get("text"));
        adminItems.add(item);
        return item;
    }

    // ---- ADMIN ONLY : PUT /api/admin/data/{id} ----
    @PutMapping("/api/admin/data/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String updateAdminItem(@PathVariable int id, @RequestBody Map<String, String> body) {
        return "Admin item " + id + " updated to: " + body.get("text");
```

```
    }

    // ---- ADMIN ONLY : DELETE /api/admin/data/{id} ----
    @DeleteMapping("/api/admin/data/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteAdminItem(@PathVariable int id) {
        adminItems.removeIf(item -> item.get("id").equals(id));
        return "Admin item " + id + " deleted";
    }

    // ---- BOTH ROLES : GET /api/user/data ----
    @GetMapping("/api/user/data")
    @PreAuthorize("hasAnyRole('ADMIN','USER')")
    public Map<String, String> getUserData() {
        return Map.of("message", "Accessible by USER or ADMIN");
    }
}
```

Step 6 & 7 — Test both roles against both endpoint groups

1. Call GET/POST/PUT/DELETE on /api/admin/data using normaluser's credentials — expect 403 Forbidden on every verb.

2. Repeat the same calls using adminuser's credentials — expect success (200) on every verb.

3. Call GET /api/user/data with BOTH sets of credentials and confirm both succeed.

Postman API Testing

Practical 17 — Postman API Testing (Role-Based Access Control)

▸ 1. GET admin-only data with USER role → 403 Forbidden

GET `http://localhost:8080/api/admin/data` **Send**

Authorization: Basic Auth (normaluser : password) — ROLE_USER

Response **Status: 403 Forbidden**

```
{
  "error": "Forbidden",
  "message": "Access is denied",
  "status": 403
}
```

▸ 2. GET admin-only data with ADMIN role → 200 OK

GET `http://localhost:8080/api/admin/data` **Send**

Authorization: Basic Auth (adminuser : password) — ROLE_ADMIN

Response **Status: 200 OK**

```
{
  "message": "Welcome Admin, here is the sensitive data"
}
```

▸ 3. GET shared data with either role → 200 OK

GET `http://localhost:8080/api/user/data` **Send**

Authorization: Basic Auth (normaluser : password) — ROLE_USER

Response **Status: 200 OK**

```
{
  "message": "Accessible by USER or ADMIN"
}
```

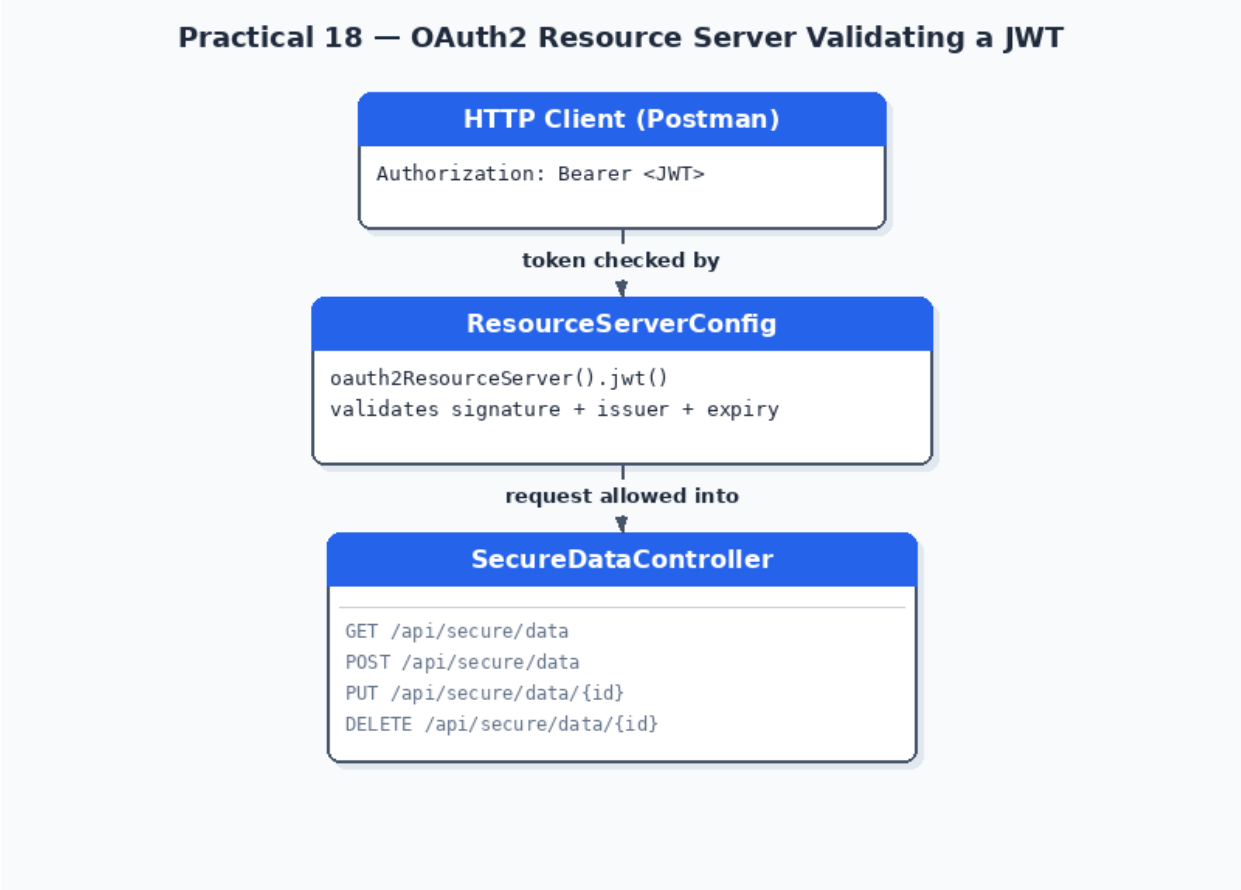
Practical No. 18 — Securing REST API as an OAuth2 Resource Server with JWT

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To configure the Spring Boot application as an OAuth2 Resource Server that validates JSON Web Tokens (JWT) issued by an Authorization Server, protecting REST endpoints so that only requests carrying a valid Bearer token succeed.

Design — JWT Validation Flow



> ℹ *Unlike Practicals 15-17, this Spring Boot application does NOT issue tokens or check a username/password itself — it only VALIDATES tokens issued elsewhere by a separate Authorization Server. You will need a running Authorization Server (e.g. Keycloak, Okta, or any OIDC provider your institution provides) to obtain a real test token.*

Step-by-Step Implementation

Step 1 — Add spring-boot-starter-oauth2-resource-server

pom.xml (add this dependency)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Step 2 — Point at your Authorization Server

application.properties (append)

```
# ==== OAuth2 Resource Server (Practical 18) ====
# Points at the Authorization Server that issues/validates the JWTs.
# For a quick classroom demo you can run a free JWT issuer such as a
# local Keycloak instance, or any Authorization Server your college
# provides; jwk-set-uri is that server's public-key endpoint.
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:9000
# -- OR, if you only have a JWK Set URL --
# spring.security.oauth2.resourceserver.jwt.jwk-set-uri=http://localhost:9000/.well-known/jwks.json
```

Step 3 — ResourceServerConfig

src/main/java/com/exam/p18/ResourceServerConfig.java

```
package com.exam.p18;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/secure/**").authenticated()
                .anyRequest().permitAll()
            )
            // Validates any incoming Authorization: Bearer <JWT> header:
            // checks the signature against the issuer's public key, and
            // checks the token has not expired.
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {}))
            .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

Step 4 — Obtain a sample JWT

Use your Authorization Server's token endpoint (e.g. via Postman's OAuth 2.0 authorization tab, or a client-credentials curl request) to obtain a signed access token for testing.

Step 5, 6 & 7 — SecureDataController with scope checks

src/main/java/com/exam/p18/SecureDataController.java

```
package com.exam.p18;

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {

    private final List<Map<String, Object>> items = new CopyOnWriteArrayList<>();
```

```
    private final AtomicInteger idCounter = new AtomicInteger(1);

    // ---- GET : requires ANY valid JWT ----
    @GetMapping("/data")
    public Map<String, String> getSecureData() {
        return Map.of("message", "JWT verified - here is your protected data");
    }

    // ---- POST : optionally requires a specific 'write' scope in the token ----
    @PostMapping("/data")
    @PreAuthorize("hasAuthority('SCOPE_write')")
    public Map<String, Object> createSecureData(@RequestBody Map<String, String> body) {
        Map<String, Object> item = Map.of("id", idCounter.getAndIncrement(), "text", body.get("text"));
        items.add(item);
        return item;
    }

    // ---- PUT : same 'write' scope requirement ----
    @PutMapping("/data/{id}")
    @PreAuthorize("hasAuthority('SCOPE_write')")
    public String updateSecureData(@PathVariable int id, @RequestBody Map<String, String> body) {
        return "Item " + id + " updated to: " + body.get("text");
    }

    // ---- DELETE : same 'write' scope requirement ----
    @DeleteMapping("/data/{id}")
    @PreAuthorize("hasAuthority('SCOPE_write')")
    public String deleteSecureData(@PathVariable int id) {
        items.removeIf(item -> item.get("id").equals(id));
        return "Item " + id + " deleted";
    }
}
```

Test all three outcomes for every verb

1. Call each endpoint with NO Authorization header → 401 Unauthorized.

2. Call each endpoint WITH a valid Bearer token (and, for POST/PUT/DELETE, the write scope) → success.

3. Call each endpoint with a deliberately corrupted or expired token → 401 Unauthorized with an invalid_token error.

Postman API Testing

Practical 18 — Postman API Testing (OAuth2 Resource Server / JWT)

▸ 1. GET secure data WITHOUT a token → 401 Unauthorized

GET http://localhost:8080/api/secure/data Send

Response Status: 401 Unauthorized

```
{
  "error": "invalid_token",
  "message": "Full authentication is required"
}
```

▸ 2. GET secure data WITH a valid Bearer JWT → 200 OK

GET http://localhost:8080/api/secure/data Send

Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...(valid)

Response Status: 200 OK

```
{
  "message": "JWT verified — here is your protected data"
}
```

▸ 3. GET secure data WITH an expired/invalid token → 401 Unauthorized

GET http://localhost:8080/api/secure/data Send

Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...(expired)

Response Status: 401 Unauthorized

```
{
  "error": "invalid_token",
  "message": "Jwt expired at 2026-08-20T10:00:00Z"
}
```

> *i In Postman, paste the token into the Authorization tab as Bearer Token, or manually add an Authorization: Bearer <token> header.*

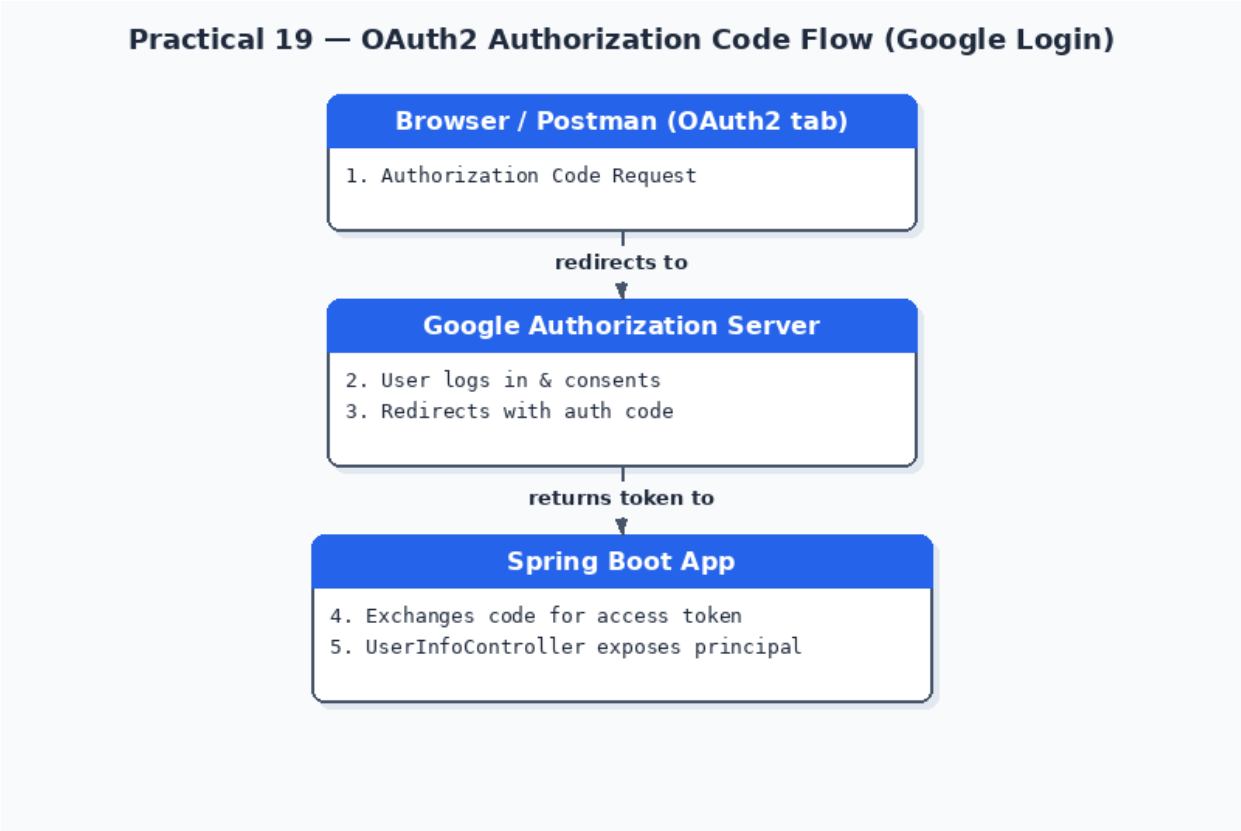
Practical No. 19 — OAuth2 Login with a Third-Party Provider (Google) and Accessing a Secured REST API

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To implement the OAuth2 Authorization Code grant using Spring Security's OAuth2 Client support with a real provider (Google), obtain an access token, and use that token to call a secured REST endpoint.

Design — Authorization Code Flow



Step-by-Step Implementation

Step 1 — Register an OAuth application with Google

1. Go to the Google Cloud Console → APIs & Services → Credentials → Create OAuth Client ID.
2. Set the Authorized redirect URI to http://localhost:8080/login/oauth2/code/google.
3. Note down the generated Client ID and Client Secret.

Step 2 — Add spring-boot-starter-oauth2-client

pom.xml (add this dependency)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Step 3 — Register Google as a client in application.properties

application.properties (append)

```
# ==== OAuth2 Client - Google Login (Practical 19) ====
spring.security.oauth2.client.registration.google.client-id=YOUR_GOOGLE_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_GOOGLE_CLIENT_SECRET
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

Step 3 (contd.) — Enable OAuth2 login in SecurityConfig

src/main/java/com/exam/p19/OAuth2ClientSecurityConfig.java

```
package com.exam.p19;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class OAuth2ClientSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/error").permitAll()
                .anyRequest().authenticated()
            )
            // Enables Spring Security's built-in OAuth2 Authorization Code
            // flow: visiting /oauth2/authorization/google redirects the
            // browser to Google's login/consent screen automatically.
            .oauth2Login(oauth2Login -> {});
        return http.build();
    }
}
```

Step 4 & 6 — UserInfoController with full CRUD notes

src/main/java/com/exam/p19/UserInfoController.java

```
package com.exam.p19;

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

@RestController
public class UserInfoController {

    private final List<Map<String, Object>> notes = new CopyOnWriteArrayList<>();
    private final AtomicInteger idCounter = new AtomicInteger(1);

    // ---- GET /api/user/profile : reads attributes straight off the OAuth2User ----
    @GetMapping("/api/user/profile")
    public Map<String, Object> getUserProfile(@AuthenticationPrincipal OAuth2User principal) {
        return Map.of(
                "name", principal.getAttribute("name"),
                "email", principal.getAttribute("email"));
    }
```

```
    // ---- POST /api/user/notes : any logged-in OAuth2 user may create notes ----
    @PostMapping("/api/user/notes")
    public Map<String, Object> createNote(@AuthenticationPrincipal OAuth2User principal,
                                          @RequestBody Map<String, String> body) {
        Map<String, Object> note = Map.of(
                "id", idCounter.getAndIncrement(),
                "text", body.get("text"),
                "owner", principal.getAttribute("email"));
        notes.add(note);
        return note;
    }

    // ---- PUT /api/user/notes/{id} ----
    @PutMapping("/api/user/notes/{id}")
    public String updateNote(@PathVariable int id, @RequestBody Map<String, String> body) {
        return "Note " + id + " updated to: " + body.get("text");
    }

    // ---- DELETE /api/user/notes/{id} ----
    @DeleteMapping("/api/user/notes/{id}")
    public String deleteNote(@PathVariable int id) {
        notes.removeIf(n -> n.get("id").equals(id));
        return "Note " + id + " deleted";
    }
}
```

Step 5 — Complete the browser login flow once, manually

Browse to http://localhost:8080/oauth2/authorization/google, sign in with a Google account, grant consent, and confirm you are redirected back to the app with an authenticated session.

Step 7 & 8 — Test via Postman's built-in OAuth 2.0 support

1. In Postman's Authorization tab, set Type = OAuth 2.0, then fill in the Auth URL (https://accounts.google.com/o/oauth2/v2/auth), Token URL (https://oauth2.googleapis.com/token), Client ID, Client Secret, and the same Redirect URI, then click Get New Access Token.

2. Call GET /api/user/profile with the retrieved token and confirm your name/email are returned.

3. Reuse the same token to call POST, PUT and DELETE on /api/user/notes and confirm each succeeds while the token remains valid.

Postman API Testing

Practical 19 — Postman API Testing (OAuth2 Login with Google)

▸ **1. Postman Authorization tab → OAuth 2.0 → Get New Access Token (Google)**

`GET` `http://localhost:8080/api/user/profile` **Send**

Authorization: Bearer <access_token from Google OAuth2 flow>

Response **Status: 200 OK**

```
{
  "name": "Aarav Mehta",
  "email": "aarav.mehta@gmail.com"
}
```

▸ **2. POST — create a note using the same access token**

`POST` `http://localhost:8080/api/user/notes` **Send**

Authorization: Bearer <access_token from Google OAuth2 flow>

Body > raw (JSON)

```
{
  "text": "Finish the OAuth2 practical"
}
```

Response **Status: 200 OK**

```
{
  "id": 1,
  "text": "Finish the OAuth2 practical",
  "owner": "aarav.mehta@gmail.com"
}
```

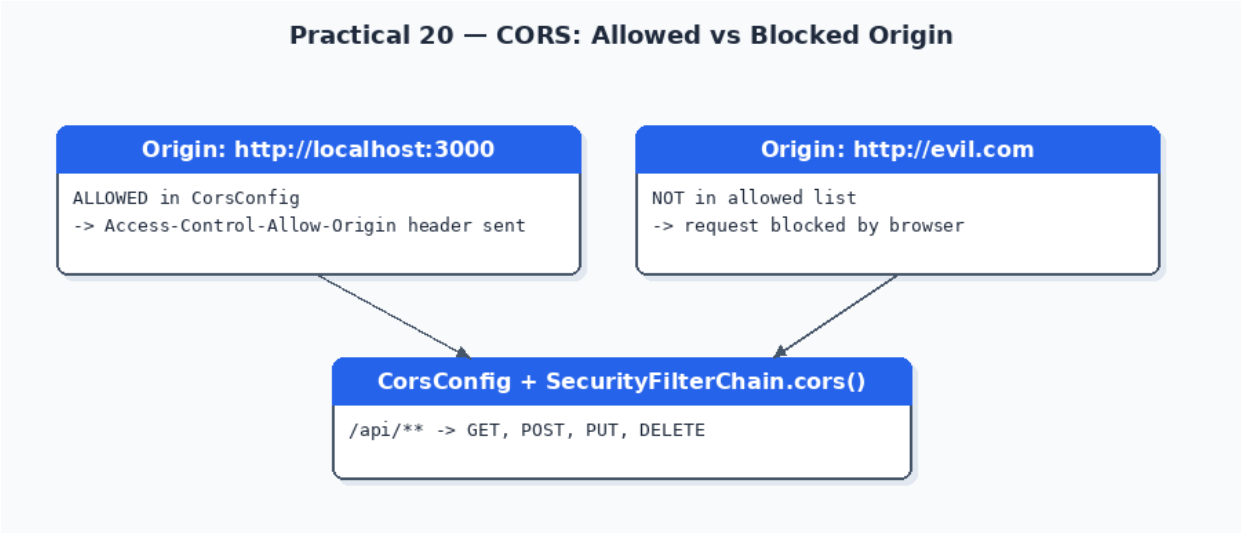
Practical No. 20 — Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs

Textbook Reference: Chapter 7 — Securing a Web Application

Objective

To configure CORS at the application level so that a secured REST API can be safely consumed by a browser-based frontend hosted on a different origin.

Design — Allowed vs Blocked Origin



> ***i*** *CORS is a BROWSER-enforced protection, not a server-enforced one. Postman itself is never blocked by CORS (it is not a browser), but Postman can still SIMULATE a cross-origin browser request by manually adding an Origin header, which is exactly how this practical is tested.*

Step-by-Step Implementation

Step 1 — CorsConfig

src/main/java/com/exam/p20/CorsConfig.java

```
package com.exam.p20;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                // Only this origin (a sample React/Angular dev server) may
                // call our API from browser JavaScript.
                .allowedOrigins("http://localhost:3000")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("*");
    }
}
```

Step 2 — Enable CORS inside SecurityConfig too

src/main/java/com/exam/p20/SecurityConfig.java

```
package com.exam.p20;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/secure/**").authenticated()
                .anyRequest().permitAll()
            )
            .httpBasic(httpBasic -> {})
            // IMPORTANT: without this line, Spring Security ignores the
            // CorsConfig bean above and blocks cross-origin calls anyway -
            // CORS must be enabled at BOTH the MVC layer and the security
            // filter chain layer.
            .cors(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable());
        return http.build();
    }
}
```

A simple secured resource to test against

src/main/java/com/exam/p20/SecureDataController.java

```
package com.exam.p20;

import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/secure")
public class SecureDataController {

    @GetMapping("/data")
    public Map<String, String> getData() {
        return Map.of("message", "data ok");
    }

    @PostMapping("/data")
    public Map<String, String> createData(@RequestBody Map<String, String> body) {
        return Map.of("message", "created: " + body.get("text"));
    }

    @PutMapping("/data/{id}")
    public Map<String, String> updateData(@PathVariable int id, @RequestBody Map<String, String> body) {
        return Map.of("message", "updated " + id + ": " + body.get("text"));
    }

    @DeleteMapping("/data/{id}")
    public Map<String, String> deleteData(@PathVariable int id) {
        return Map.of("message", "deleted " + id);
    }
}
```

Step 3, 4 & 5 — Simulate cross-origin calls from Postman for every verb

1. In Postman, open the Headers tab of a GET request to /api/secure/data and manually add a header: Origin = http://localhost:3000.

2. Send the request and inspect the RESPONSE HEADERS (not the body) — look for Access-Control-Allow-Origin: http://localhost:3000.

3. Repeat the same Origin header test for POST, PUT and DELETE /api/secure/data and confirm the header appears on all four verbs.

Step 6 — Test a disallowed origin

Change the Origin header to http://evil.com and resend — the Access-Control-Allow-Origin header should now be ABSENT from the response (a real browser would then block the frontend JavaScript from reading the response, even though Postman itself still shows you the raw response).

Step 7 — Same-origin vs cross-origin, in your own words

- A same-origin Postman call (no Origin header at all) never triggers CORS checks — the browser-security concept simply does not apply outside a browser context.
- A simulated cross-origin call (Origin header present) exercises the exact header-based negotiation a real browser performs before allowing JavaScript on a different domain to read the response.

Postman API Testing

Practical 20 — Postman API Testing (CORS: Allowed vs Blocked Origin)

▸ 1. GET with Origin: http://localhost:3000 (allowed) → CORS header present

GET `http://localhost:8080/api/secure/data` **Send**

Authorization: Origin: http://localhost:3000

Response **Status: 200 OK**

```
{
  "message": "data ok",
  "_headers": {
    "Access-Control-Allow-Origin": "http://localhost:3000"
  }
}
```

▸ 2. POST with Origin: http://localhost:3000 (allowed)

POST `http://localhost:8080/api/secure/data` **Send**

Authorization: Origin: http://localhost:3000

Body > raw (JSON)

```
{
  "text": "cross-origin create"
}
```

Response **Status: 200 OK**

```
{
  "id": 2,
  "text": "cross-origin create",
  "_headers": {
    "Access-Control-Allow-Origin": "http://localhost:3000"
  }
}
```

▸ 3. GET with Origin: http://evil.com (NOT allowed) → blocked

GET `http://localhost:8080/api/secure/data` **Send**

Authorization: Origin: http://evil.com

Response **Status: 403 Forbidden**

```
{
  "error": "CORS request rejected – origin not allowed"
}
```